De la Géométrie de l'Interaction à la Syntaxe Transcendantale

Sambo Boris ENG, supervisé par Thomas Seiller (LoVe, LIPN)

Rapport de stage de M2 MPRI

Le contexte général

La correspondance de Curry-Howard [Cur34 ; How80] établit l'existence d'une intersection entre logique et calcul en remettant en question leurs fondements autant d'un point de vue technique que philosophique. Logique et calcul sont tout d'abord reliés de façon structurelle par une correspondance formule/type et preuve/programme mais aussi de façon dynamique car l'évaluation des programmes se comporte comme une procédure de la logique appelée *élimination des coupures* [Gen35a ; Gen35b].

Dans cette optique, la logique linéaire [Gir87a], principalement initiée par Jean-Yves Girard nous permet d'étudier le contenu opérationnel de la logique : les preuves, traditionnellement définies comme des arbres d'applications de règles de déduction deviennent des *réseaux de preuve* exhibant une structure plus précise des preuves. Le programme de « Géométrie de l'Interaction » [Gir89b ; Gir89a ; Gir88 ; Gir95 ; Gir06 ; Gir11 ; Gir13a] succédant à la logique linéaire propose de se libérer du format des réseaux pour étudier ce qui les caractérise mathématiquement. C'est un modèle de la logique linéaire qui diffère des autres par la considération d'une dynamique d'exécution alors que les modèles dénotationnels vont se concentrer sur l'aspect observationnel.

$$\begin{tikzcd}
\pi \arrow[r, "\text{traduction}"] \arrow[d, "\text{coupures}"'] & \|\pi\| \arrow[d, phantom, "="] \\
\pi' \arrow[r, dotted, "\text{traduction}"'] & \|\pi'\|
\end{tikzcd}$$

Figure 1 – Modèle dénotationnel

$$\begin{tikzcd}
\pi \arrow[r, "\text{traduction}"] \arrow[d, "\text{coupures}"'] & \|\pi\| \arrow[d, "\text{exécution}"] \\
\pi' \arrow[r, dotted, "\text{traduction}"'] & \|\pi'\|
\end{tikzcd}$$

Figure 2 – Géométrie de l'Interaction

Les modèles dénotationnels (figure 1) aplatissent l'interprétation d'une preuve et son évaluation par élimination des coupures car ils représentent en un certain sens la même entité (du point de vue subjectif). Le modèle de géométrie de l'interaction (figure 2), en prenant en compte la présence d'une notion d'exécution permet une analyse fine des propriétés calculatoires de la logique qui sont précieuses pour les études de complexité.

En partant de ces travaux, Girard propose de reconstruire la logique en plaçant l'exécution comme primitive ce qui débouche sur le programme de « Syntaxe Transcendantale » [Gir13a ; Gir13b ; Gir17 ; Gir16b ; Gir16c] qui étend la géométrie de l'interaction.

Le problème étudié

Les principaux articles écrits par Girard ne posent que le plan architectural du projet sans développement technique précis. Le sujet du stage était de formaliser ce programme de Syntaxe Transcendantale. Il a fallu établir clairement les définitions, rédiger les théorèmes et les prouver.

Son intérêt technique repose sur une inversion méthodologique : au lieu d'explorer les propriétés de la logique définie comme telle, on introduit un vaste espace calculatoire d'où la logique émerge. Cette émergence repose sur une dualité entre les entités de notre espace qui leur permettent d'interagir par une procédure calculatoire. Cette approche pourrait entre autre permettre de donner un contenu calculatoire à la logique du premier ordre [Gir16c] (qui n'est pas capturée par la correspondance de Curry-Howard et qui a donc un statut purement logique) et donner une relecture des résultats de complexité descriptive en plus de d'autres applications à la complexité à rapprocher à ceux des flots [Bag14 ; BP01 ; AB14 ; ABS16], des graphes d'interaction [Sei18 ; Sei13].

La contribution proposée

Le stage a apporté un premier pas dans la formalisation de la Syntaxe Transcendantale. Le modèle de calcul introduit par le programme (étoiles et constellations) a été décrit formellement et rapproché des modèles de flots [Gir95 ; Bag14] et des graphes d'interaction [Sei12b]. En particulier, le cheminement de la géométrie de l'interaction à la conception de la syntaxe transcendantale est décrit. Comme premier pas, un modèle du fragment multiplicatif de la logique linéaire (MLL) est donné.

La démarche scientifique avait un fort penchant expérimental : il était nécessaire de construire nous-même les outils à utiliser, produisant parfois des échecs dûs à une formalisation mal fondée. Nous sommes partis d'une généralisation du modèle de Girard et avons étudié ses propriétés calculatoires afin de déterminer les contraintes nécessaires à imposer pour avoir un calcul cohérent.

Les arguments en faveur de sa validité

La formalisation repose sur une architecture générale donnée par le programme de géométrie de l'interaction et reste donc libre mais guidée par la volonté d'avoir certaines propriétés majeures postulant une certaine forme de cohérence calculatoire (confluence, associativité de l'élimination des coupures) et les propriétés nécessaires pour pouvoir reconstruire un fragment minimal de la logique.

Le bilan et les perspectives

Ce travail apporte une ouverture aux derniers développements de la géométrie de l'interaction qui disposent de très peu d'écrits mais constitue aussi une première introduction technique au programme de Syntaxe Transcendantale.

L'objectif suivant serait de modéliser des fragments plus riches de la logique linéaire (notamment le fragment additif MALL de la logique linéaire qui est sujet à des problèmes techniques) dans l'espoir de mieux les comprendre. Une contribution intéressante serait de donner un modèle de la logique du premier ordre (comme esquissée par Girard [Gir16c]) avec la complexité en ligne de mire. Il serait aussi intéressant de trouver un modèle mathématique des étoiles et des constellations qu'on pourrait comparer aux graphages [Sei17].

1 Architecture

Nous commençons par présenter la base et la position du projet de Syntaxe Transcendantale dans l'évolution du domaine de la logique mathématique. Nous prenons racine dans la *théorie de la démonstration* qui est une branche de la logique plaçant la preuve mathématique à la position d'objet d'étude à part entière.

1.1 Traditions

Traditionellement, on considère que le raisonnement est régit par des *lois logiques* dont les formulations sont multiples. On fait le choix d'utiliser le *calcul des séquents* de Gentzen [Gen35a ; Gen35a].

Un séquent est de la forme $A_1, ..., A_n \vdash B_1, ..., B_k$ et énonce que l'*une* (disjonction $\vee$) des conclusions $B_{1 \leq i \leq k}$ est prouvable sous *toutes* (conjonction $\wedge$) les hypothèses $A_1, ..., A_n$. Les entités $A_1, ..., A_n, B_1, ..., B_k$ sont des *formules* qui encodent des faits qui sont qualifiés de *vrais* ou *faux*. Une règle d'inférence est de la forme $\frac{\Gamma' \vdash \Delta'}{\Gamma \vdash \Delta}$ où $\Gamma, \Gamma', \Delta, \Delta'$ sont des multi-ensembles de formules.

La théorie de la démonstration se concentre sur le mécanisme calculatoire des *systèmes de déduction* c'est-à-dire la production de preuves mathématiques par application successive de règles fixées. Par exemple, on a les règles suivantes :

$$\frac{}{A \vdash A}\, ax \qquad \frac{\Gamma, A, B \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta}\, \wedge_L \qquad \frac{\Gamma \vdash A, \Delta \quad \Gamma' \vdash B, \Delta'}{\Gamma, \Gamma' \vdash A \wedge B, \Delta, \Delta'}\, \wedge_R \qquad \frac{\Gamma, A \vdash B, \Delta}{\Gamma \vdash A \Rightarrow B, \Delta}\, \Rightarrow_R$$

permettent de construire la preuve suivante de l'implication $\vdash (A \wedge B) \Rightarrow (B \wedge A)$:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{B \vdash B}\,\text{ax} \qquad \dfrac{}{A \vdash A}\,\text{ax}}{A, B \vdash B \wedge A}\,\wedge_R}{A \wedge B \vdash B \wedge A}\,\wedge_L}{\vdash (A \wedge B) \Rightarrow (B \wedge A)}\,\Rightarrow_R$$

La règle (ax) pour "axiome" connecte une occurrence d'une formule A en hypothèse et une autre en conclusion. La règle $(\Rightarrow)$ introduit une hypothèse et la règle $(\wedge_R)$ nous demande de prouver A et B en distribuant les hypothèses et conclusions.

1.2 Vers une morphologie de la preuve

En 1986, Jean-Yves Girard introduit la *logique linéaire* [Gir87a] non pas comme un nouveau système logique mais comme un affinement de la logique traditionnelle. L'appelation « linéaire » provient du fait que ce système consomme exactement *une fois* chaque hypothèse ce qui nous permet de l'intérpréter comme une logique de gestion des ressources, des actions ou encore des communications [Gir89b].

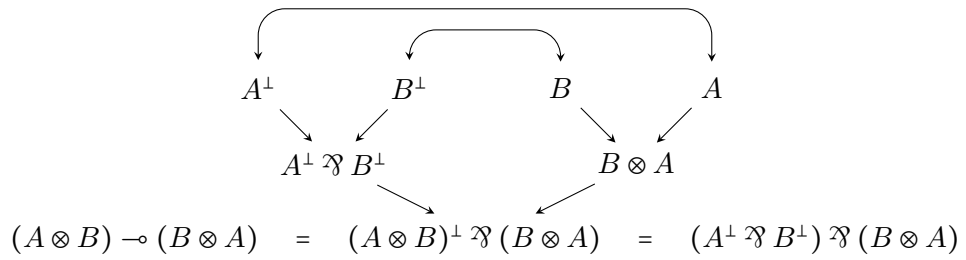
En particulier, il révèle une opérationnalité interne à la logique par une décomposition des connecteurs usuels :

$\wedge$	$\vee$	$A \Rightarrow B$
$\otimes, \&$	$\oplus, \parr$	$!A \multimap B$

où $A \multimap B$ est une implication linéaire plus primitive qui consomme son hypothèse et $!A$ représente un stock arbitraire d'hypothèses A, c'est-à-dire qu'on spécifie explicitement que $A \Rightarrow B$ peut potentiellement utiliser plusieurs fois son hypothèse (ou pas du tout) : on obtient une décomposition de $A \Rightarrow B$ en $!A \multimap B$.

En travaillant avec la logique linéaire, Girard remarque que la linéarité dévoile une structure interne des preuves. En particulier, la négation linéaire étant involutive ($A^{\perp\perp} = A$), les règles $\frac{\Gamma, A^{\perp} \vdash \Delta}{\Gamma \vdash A, \Delta}$ et $\frac{\Gamma \vdash A^{\perp}, \Delta}{\Gamma A, \vdash \Delta}$ nous permettent de transformer toutes les hypothèses en conclusion et les règles sont vues comme des moyens de connecter les formules. De plus, l'ordre d'application des règles est souvent inessentiel.

Pour exhiber cette structure interne des preuves, Girard introduit une syntaxe alternative : les *réseaux de preuve* prenant la forme d'une sorte de câblage entre des formules. Par exemple, on pourrait avoir le réseau de preuve suivant pour $(A \otimes B) \multimap (B \otimes A)$ qui est traduit en $(A^\perp \parr B^\perp) \parr (B \otimes A)$ par la traduction de l'implication $A \multimap B = A^\perp \parr B$ et la loi de Morgan $(A \otimes B)^\perp = A^\perp \parr B^\perp$:



Les réseaux de preuve forment un support pour une étude de la logique, de ses mécanismes et de ses phénomènes à travers un point de vue particulièrement riche et fin.

1.3 La dynamique de la logique

Les travaux de Gentzen [Gen35a ; Gen35b] révèlent l'existence d'une dynamique dans la logique par le « théorème d'élimination des coupures ».

La règle de *coupure* formulée par :

$$\text{version classique :} \quad \frac{\Gamma \vdash A, \Delta \quad \Gamma', A \vdash \Delta'}{\Gamma, \Gamma' \vdash \Delta, \Delta'}\ \text{cut} \qquad\qquad \text{version unilatérale :} \quad \frac{\vdash \Gamma, A \quad \vdash \Delta, A^\perp}{\vdash \Gamma, \Delta}\ \text{cut}$$

représente l'usage d'un lemme A en mathématique (qu'on prouve puis utilise) afin de continuer notre preuve. De façon plus terre-à-terre, on a simplement connecté deux preuves par anihilation de deux occurrences opposées/duales de A.

Le théorème d'élimination des coupures nous dit qu'il est possible d'éliminer l'usage des règles de coupure et qu'on peut donc démontrer sans usage de lemmes. Ce n'est pas si étonnant si on considère que l'on peut faire de la programmation sans fonctions auxiliaires, de façon irréfléchie et explicite.

Ces travaux ayant été introduits pour des soucis de cohérence de l'arithmétique, le théorème d'élimination des coupures a été complètement sous-estimé au temps de Gentzen : il donne une procédure d'évaluation des preuves qu'on peut apparenter à l'exécution des programmes en informatique et prend un statut fondamental à travers la correspondance de Curry-Howard.

C'est dans l'étude de cette procédure d'élimination des coupures que se fonde le programme de « géométrie de l'interaction » pour étudier ce qu'on appellera maintenant la *dynamique de la logique*. La logique n'est plus primitive mais sujet d'étude : on parle de mathématique de la logique.

2 Géométrie de l'Interaction

Le programme de Géométrie de l'Interaction propose une reconstruction de la logique avec les ingrédients suivants : (1) une entité généralisant les preuves (2) une notion de dualité entre ces entités (3) une notion d'exécution permettant de faire interagir deux entités duales. Une telle reconstruction nous permet de faire émerger les opérations logiques à travers leur potentiel d'interaction calculatoire. Ce programme porte aussi la volonté de libérer la logique des contraintes des formats qu'on peut lui imposer : on se place entre syntaxe et sémantique.

Pour simplifier les choses au maximum, nous travaillerons sur le fragment multiplicatif[1] de la logique linéaire appelé MLL [Gir87b ; DR89] (pour *multiplicative linear logic*) avec les formules définies par :

$$A, B \quad := \quad a \mid A^\perp \mid A \otimes B \mid A \parr B$$

où $\otimes$ est appelé *tenseur*, $\parr$ est appelé *par* et a est une formule minimale appelée *atome*. On peut reconstituer l'implication linéaire par $A \multimap B := A^\perp \parr B$. On dispose des règles suivantes du calcul des séquents linéaire :

$$\frac{}{\vdash A, A^\perp}\ ax \qquad \frac{\vdash \Gamma, A \quad \vdash \Delta, A^\perp}{\vdash \Gamma, \Delta}\ cut \qquad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B}\ \otimes \qquad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \parr B}\ \parr$$

où l'ordre des formules n'importe pas dans un séquent (car considéré comme un multi-ensemble). Conformément aux lois de De Morgan qui sont aussi prouvables dans le cadre linéaire, on a

$$(A \otimes B)^\perp = A^\perp \parr B^\perp \qquad (A \parr B)^\perp = A^\perp \otimes B^\perp \qquad A = A^{\perp\perp}$$

qui nous permettent de ne considérer la négation que sur les atomes et de pouvoir travailler sur une grammaire plus simple des formules :

$$A, B \quad := \quad a \mid a^\perp \mid A \otimes B \mid A \parr B$$

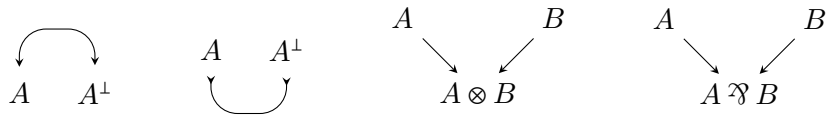
où a est un atome dans un ensemble dénombrable d'atomes qu'on a fixé.

2.1 Réseaux de preuve de MLL

En se libérant de la syntaxe arborescente, on obtient des objets plus généraux que les preuves, qu'on peut voir comme des entités calculatoires (du même genre que les programmes) : les structures de preuve.

Definition 1 (Structure de preuve).

Une structure de preuve est un hypergraphe orienté avec des formules comme sommets et les hyperarêtes (arêtes avec plusieurs sources et cibles) suivantes :

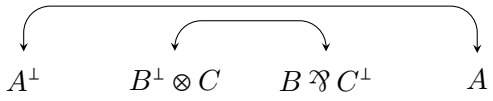


où chaque sommet (formule) est but d'exactement une hyperarête. La première et la seconde hyperarête représentent respectivement l'axiome et la coupure. Les cibles d'hyperarête qui ne sont pas sources sont appelées *conclusions*.

Definition 2 (Véhicule).

Le **véhicule** $V(\pi)$ d'une structure de preuve π est l'ensemble de ses axiomes.

Exemple.



1. Note aux spécialistes : on fait le choix d'éviter de parler des unités $\perp$ et 1

Une conséquence de cette métamorphose des entités logiques est qu'on perd la distinction comportementale entre $\otimes$ et $\parr$ que l'on doit retrouver par un critère. Un tel critère est appelé *critère de correction.*

Definition 3 (Test).

Soit π une structure de preuve et $\varphi : \mathcal{P}(\pi) \longrightarrow \{\mathtt{l}, \mathtt{r}\}$ une sélection gauche/droite pour chaque sommet $\parr$ de π. Le *test* $\mathtt{test}_\varphi(\pi)$ induit par π et φ est défini de la façon suivante où p désigne le sommet $\parr$ concerné :

Si $\phi(p) = \mathtt{l}$ alors
$$\begin{tikzcd} A \arrow[dr, dash] & & B \\ & A \parr_L B & \end{tikzcd}$$

Si $\phi(p) = \mathtt{r}$ alors
$$\begin{tikzcd} A & & B \arrow[dl, dash] \\ & A \parr_R B & \end{tikzcd}$$

où dans les autres cas, les arêtes deviennent non-orientées et $\mathcal{P}(\pi)$ est l'ensemble des sommets $\parr$ dans la preuve π. Les tests, destinés à être connectés à un véhicule, permettent de définir une preuve par un ensemble véhicule+test qui définit un arbre (pour se ramener à la forme arborescente du calcul des séquents).

Exemple. On a les deux tests suivants pour le réseau de preuve de $(A \otimes B) \multimap (B \otimes A)$ introduit précédemment :

$$\begin{tikzcd} A^\perp \arrow[dr, dash] & & B^\perp \arrow[dl, dash] & & B \arrow[dr, dash] & & A \arrow[dl, dash] \\ & A^\perp \parr_R B^\perp \arrow[drr, dash] & & & & B \otimes A \arrow[dl, dash] & \\ & & & (A^\perp \parr B^\perp) \parr_L (B \otimes A) & & & \end{tikzcd}$$

$$\begin{tikzcd} A^\perp \arrow[dr, dash] & & B^\perp & & B \arrow[dr, dash] & & A \arrow[dl, dash] \\ & A^\perp \parr_L B^\perp \arrow[drr, dash] & & & & B \otimes A \arrow[dl, dash] & \\ & & & (A^\perp \parr B^\perp) \parr_L (B \otimes A) & & & \end{tikzcd}$$

Definition 4 (Gabarit).

Le **gabarit** $G(\pi)$ d'une structure de preuve π est l'ensemble de tous les tests $\mathtt{test}_\varphi(\pi)$ pour tous les choix possibles de φ. Il y en a exactement $|\{l, r\}|^{|\mathcal{P}(\pi)|} = 2^{|\mathcal{P}(\pi)|}$.

Definition 5 (Critère de correction de Danos-Regnier).

Soit π une structure de preuve de MLL. Pour tout test $t \in G(\pi), V(\pi)+t$ est un arbre où $V(\pi)+t$ est t que l'on a complété avec les axiomes de π.

Théorème 1 (Séquentialisation).

Si le critère de correction de Danos-Regnier est satisfait par une structure de preuve π de MLL alors π est la traduction d'une preuve du calcul des séquents de MLL sachant une définition naturelle du calcul des séquents vers les structures de preuve (qu'on omet ici).

Preuve.

On se réfèrera à la thèse de Laurent Regnier [Reg92] qui propose une preuve de séquentialisation avec l'introduction d'un critère de correction à base d'interrupteurs correspondant à ce qu'on appelle *test* dans ce document.

□

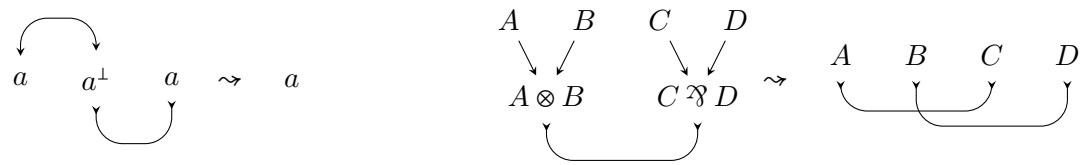
Definition 6 (Réseau de preuve).

Un *réseau de preuve* (ou simplement *réseau*) est une structure de preuve qui satisfait le critère de correction de Danos-Regnier.

Une structure de preuve est une entité qui obtient le titre (subjectif) de "réseau de preuve" seulement si elle passe un ensemble de tests avec lesquels elle intéragit. De cette idée, Girard introduit une terminologie d'industrie automobile qu'on a choisi d'adopter : on certifie l'usage d'un véhicule s'il passe un ensemble de tests proposés par une usine. C'est une analogie qui rappelle aussi les *tests unitaires* en programmation.

Definition 7 (Elimination des coupures).

On définit une procédure d'élimination des coupures qui va permettre de réduire/évaluer les structures de preuves vers d'autres structures de preuves équivalentes :



2.2 Caractérisation algébrique

Nous faisons le choix de partir de l'algèbre Λ^* de Girard [Gir89a] pour proposer une caractérisation algébrique des réseaux qui nous mènera aux flots que nous généraliserons avec les étoiles et constellations.

Definition 8 (Chemin).

Comme les structures de preuve sont des hypergraphes, on a quelques difficultés pour définir une notion de chemin.

Un **chemin** φ dans une structure de preuve de MLL π est une séquence $\varphi = v_0 e_1 v_1 ... e_n v_n$ où les v_i sont des sommets de π (formules) et les e_i sont soit des hyperarêtes ou des sources/cibles d'hyperarêtes. C'est-à-dire qu'on décrit comment on passe d'un sommet à un autre (soit par un axiome ou une coupure soit une prémisse/conclusion de connecteur binaire). Pour suivre les conventions de la littérature, on écrit les chemins de droite à gauche.

Definition 9 (Monoïde involutif L^*).

On définit L^* (Λ^* chez Girard) comme étant le monoïde involutif généré par $\{\mathtt{l}, \mathtt{r}, 0\}$ sujet aux relations suivantes :

$$0x = x0 = 0 \qquad \mathtt{l}^*\mathtt{r} = \mathtt{r}^*\mathtt{l} = 0 \qquad \mathtt{l}^*\mathtt{l} = \mathtt{r}^*\mathtt{r} = 1$$

L'involution est une opération unaire (plus précisément, un antimorphisme) définie par $0^* = 0$, $(w^*)^* = w$ et $(vw)^* = w^* v^*$.

Exemple. on peut avoir $\mathtt{lr}^*\mathtt{rr}^* = \mathtt{lr}^*$ par la relation $\mathtt{r}^*\mathtt{r} = 1$.

Definition 10 (Pondération).

On définit une fonction de **poids** $\omega(\varphi)$ sur un chemin $\varphi = v_0 e_1 v_1 ... e_n v_n$ d'une structure de preuve de MLL avec $\omega(v_0 e_1 v_1 ... e_n v_n) = \omega(v_0)\omega(e_1)\omega(v_1)...\omega(e_n)\omega(v_n)$ et :

— $\omega(e_i) = \mathtt{l}$ si e_i est une prémisse gauche d'un connecteur binaire

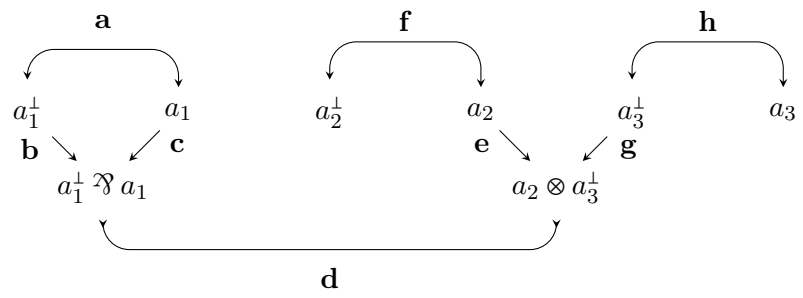
— $\omega(e_i) = \mathtt{r}$ si e_i est une prémisse droite d'un connecteur binaire
— $\omega(e_i) = 1$ si e_i est un axiome ou une coupure et $v_i \neq v_{i+1}$ (pas de demi-tour)
— $\omega(x) = 0$ sinon

On peut étendre la fonction de poids sur :
— les arêtes inverses par $w(e^R) = \omega(e)^*$
— les chemins par $\omega(e_1 \cdot \ldots \cdot e_n) = \omega(e_1) \cdot \ldots \cdot \omega(e_n)$

Definition 11 (Chemin régulier).

Un chemin φ est **régulier** si $\omega(\varphi) \neq 0$.

L'élément absorbant 0 va représenter la destruction de chemins par l'élimination des coupures. Danos et Regnier [DR95] ont montré une équivalence entre la notion de chemin régulier et de chemin droit *persistant* qui est un chemin qui ne fait pas de demi-tour, ne passe pas d'une prémisse à l'autre et survit à l'élimination des coupures. De tels chemins caractérisent la forme normale du réseau. Illustrons en quoi L^* décrit l'élimination des coupures :



Supposons qu'on écrive les chemins de droite à gauche et que $a_1 = a_2 = a_3 = a$. Pour simplifier on représente les chemins par une séquence d'étiquettes :
— En partant de a_3, on a les chemins suivants :
$\omega(fe^Rdcab^Rdgh) = \mathtt{l}^*\mathtt{rl}^*\mathtt{r} = 0$ $\quad$ $\omega(fe^Rdbac^Rdgh) = \mathtt{l}^*\mathtt{lr}^*\mathtt{r} = 1$ $\quad$ (vers $a_2^\perp$)
$\omega(hg^Rdcab^Rdgh) = \mathtt{r}^*\mathtt{rl}^*\mathtt{r} = 0$ $\quad$ $\omega(hg^Rdbac^Rdgh) = \mathtt{r}^*\mathtt{lr}^*\mathtt{r} = 0$ $\quad$ (vers a_3)
— Si on part de $a_2^\perp$, on a les chemins suivants :
$\omega(hg^Rdcab^Rdef) = \mathtt{r}^*\mathtt{rl}^*\mathtt{l} = 1$ $\quad$ $\omega(hg^Rdbac^Rdef) = \mathtt{lr}^*\mathtt{l} = 0$ $\quad$ (vers a_3)
$\omega(fe^Rdcab^Rdef) = \mathtt{l}^*\mathtt{rl}^*\mathtt{l} = 0$ $\quad$ $\omega(fe^Rdbac^Rdef) = \mathtt{l}^*\mathtt{lr}^*\mathtt{l} = 0$ $\quad$ (vers $a_2^\perp$)

Comme chemins persistants, on obtient un chemin de a_3 à $a_2^\perp$ et un autre de $a_2^\perp$ à a_3. Cette pondération est équivalente à celle d'un axiome entre a_3 et $a_2^\perp$. Conclusion : on a procédé à une élimination des coupures sans réécriture, simplement par *analyse statique* du réseau et simplifications d'expressions algébriques. C'est un point essentiel pour parler de complexité (en particulier en espace).

Definition 12 (Exécution).

On définit **l'exécution** d'un réseau π comme la somme du poids de ses chemins persistants :

$$\mathtt{Ex}(\pi) = \sum_{\varphi \in \mathtt{persistent}(\pi)} \omega(\varphi)$$

où $\mathtt{persistent}(\pi)$ est l'ensemble des chemins persistants de π c'est-à-dire que si $\varphi \in \mathtt{persistent}(\pi)$ et π se réduit en π' alors $\varphi \in \mathtt{persistent}(\mathcal{R}')$. Cette exécution représente l'évaluation de π par élimination des coupures c'est-à-dire que si π se réduit en π' alors $\mathtt{Ex}(\pi) = \mathtt{Ex}(\pi')$.

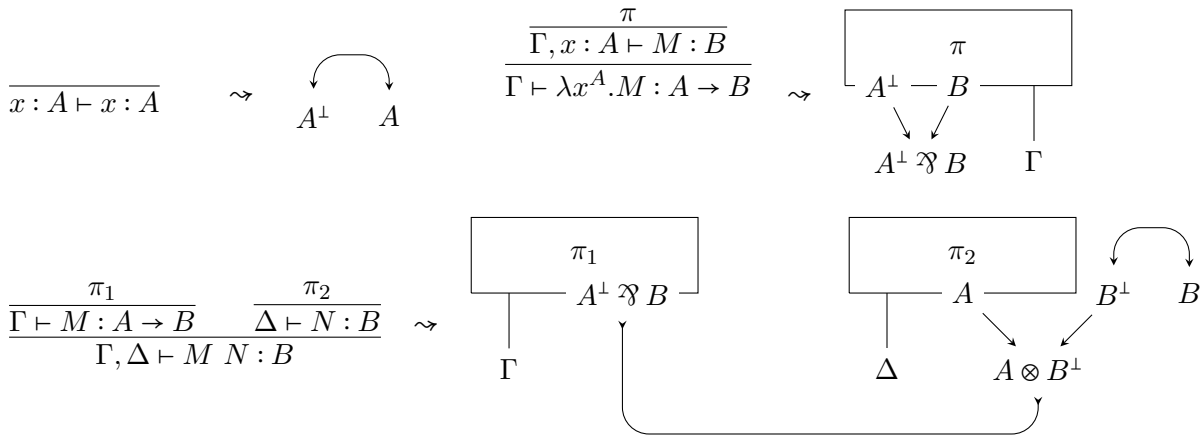
2.3 Interlude : réseaux et lambda-calcul

Pour revenir à des notions familières de l'informatique, on peut modéliser le λ-calcul linéaire et son execution par les réseaux et la géométrie de l'interaction dans MLL. Si on fixe un ensemble de variables et de types de base, la grammaire du λ-calcul simplement typé est donnée par :

$$M, N \quad := \quad x \mid \lambda x^A.M \mid M\ N \qquad\qquad A, B \quad := \quad a \mid A \to B$$

Le λ-calcul linéaire est un modèle de calcul fonctionnel où les arguments sont utilisés exactement une fois et sont donc en quelque sorte consommés à l'usage. L'exécution est donnée par la β-réduction définie par $(\lambda x.M)N \to_\beta M[x := N]$ où $M[x := N]$ est M où on a remplacé l'unique occurrence de x par N.

On peut traduire les règles de typage du λ-calcul simplement typé dans les réseaux [Reg92] :



La géométrie de l'interaction fournit en fait un moyen de lire statiquement un réseau représentant un λ-terme pour en cacluler la forme normale correspondante. En particulier, cela a un intérêt précieux en complexité [Maz17 ; Maz15 ; MT15].

Le réseau précédent sur lequel nous avons calculé l'élimination des coupures par géométrie de l'interaction représente en fait le terme linéaire $y : a \vdash (\lambda x^a.x)\ y$ qui se réduit en $x : a$ qui est bien traduit en un axiome entre a et $a^\perp$. Ce que nous venons d'illustrer s'étend au cadre non-linéaire et donc au λ-calcul simplement typé et même pur [Dan90] en interprétant le fragment exponentiel de la logique linéaire.

2.4 Eléments d'unification

Durant l'étude algébrique de Girard sur la géométrie de l'interaction, une des versions s'est stabilisée vers une algèbre d'unification [Gir95]. Girard montre que l'unification, qu'on peut attribuer à Jacques Herbrand [Her30] permet de donner un modèle de géométrie de l'interaction pour la logique linéaire complète.

On suppose la théorie de l'unification comme connue. Une annexe décrit cependant les définitions qu'on utilisera ainsi que des preuves auxquelles on se réfèrera. On présente quand même quelques définitions élémentaires.

On définit une signature $\mathcal{S} = \texttt{Vars} \uplus \texttt{Func}$ où

— $\texttt{Vars} = \{w, x, y, z, ...\}$ est un ensemble dénombrable de symboles de variables.

— $\texttt{Func} = \{f^{(n_1)}, g^{(n_2)}, h^{(n_3)}, ...\}$ est un ensemble dénombrable de symboles de fonctions accompagnés d'une fonction d'arité $\texttt{ar} : \texttt{Func} \longrightarrow \mathbb{N}$.

Les $\mathcal{S}$-termes (ou simplement *termes* en l'absence d'ambiguïté) sont générés par la grammaire suivante :

$$t, u, v, w \quad := x \quad | \quad f(t_1, ..., t_{\mathtt{ar}(f)}) \qquad\qquad (\mathcal{S}\text{-termes})$$

où $x \in \mathtt{Vars}$ et $f \in \mathtt{Func}$ pour un certain $n \in \mathbb{N}$. On note $\mathtt{vars}(t)$ l'ensemble des variables d'un $\mathcal{S}$-terme t et un terme t est clos si $\mathtt{vars}(t) = \varnothing$. Fixons la signature $\mathcal{S}$ avec $\mathtt{Vars} = \{x, z\}$, $\mathtt{Func} = \{f, h\}$ et $\mathtt{ar}(f) = 1, \mathtt{ar}(h) = 2$. Exemples : $x, f(x), h(f(x), z)$. Non-exemples : $f(g), x(f), y(f(x))$.

Definition 13 (Substitution).

Une **substitution** est une fonction $\theta = \{\!|x_1 \mapsto t_1, ..., x_n \mapsto t_n|\!\}$ associant une valeur aux variables et dont l'action s'étend aux termes. Si on définit les ensembles suivants pour θ :

— $\mathtt{dom}(\theta) = \{x_1, ..., x_n\}$
— $\mathtt{img}(\theta) = \{t_1, ..., t_n\}$

alors on peut définir l'action d'une substitution par :

$$\theta(x_i) = t_i \qquad\qquad \theta(y)_{y \in \mathtt{Vars} \setminus dom(\theta)} = y \qquad\qquad \theta(f(u_1, ..., u_k)) = f(\theta u_1, ..., \theta u_k)$$

Exemple. Si on a $\theta = \{x \mapsto f(y), z \mapsto y\}$ alors $\theta(f(x, z)) = f(f(y), y)$.

Definition 14 (Composition de substitutions).

La **composition** de deux substitutions θ_1 et θ_2 est définie par $(\theta_1 \circ \theta_2)t = \theta_1(\theta_2 t)$. On la considère associative à droite par défaut c'est-à-dire que $\theta_1 \circ \theta_2 \circ \theta_3$ est une notation pour $\theta_1 \circ (\theta_2 \circ \theta_3)$.

Definition 15 (Renommage).

Un **renommage** α est une permutation sur l'ensemble des variables qu'on représente sous la forme d'une substitution α telle que $\mathtt{dom}(\alpha) = \mathtt{img}(\alpha) \subseteq \mathtt{Vars}$. Pour tout terme t et renommage α, αt est appelé **renommage** de t.

Definition 16 (Equivalence par renommage).

Deux substitutions θ_1, θ_2 sont α-**équivalentes** noté $\theta_1 \approx_\alpha \theta_2$ si et seulement si $\theta_1 = \alpha\theta_2$ ou $\theta_2 = \alpha\theta_1$ pour un certain renommage α.

Definition 17 (Unification).

Deux termes t et u sont **unifiables** quand il existe une substitution θ telle que $\theta t = \theta u$. Une telle substitution est un **unificateur** de t et u.

Exemple. Les termes $f(x, x)$ et $f(z, h(y))$ ont pour unificateur $\theta = \{x \mapsto h(y), z \mapsto h(y)\}$ mais x et $f(x)$ n'ont pas d'unificateur commun.

Definition 18 (Unificateur principal).

La composition induit un pré-ordre sur les substitutions : $\theta \preceq \psi \iff \exists \theta'. \psi = \theta' \circ \theta$.

Un **unificateur principal** θ pour deux termes t et u est un unificateur de t et u minimal pour l'ordre défini ci-dessus. On note $\theta \sim \psi$ si et seulement si $\theta \preceq \psi$ et $\psi \preceq \theta$.

On suppose qu'on utilise un algorithme d'unification non-déterministe comme celui décrit par Martelli et Montanari [MM82] pour produire un unificateur principal noté $\mathtt{Sol}(t \doteq u)$. Il est égal à $\perp$ s'il n'est pas défini et peut être étendu à des problèmes d'unification : $\mathtt{Sol}(\{t_1 \doteq u_1, ..., t_n \doteq u_n\})$ et un unificateur pour $t_1 \doteq u_1, ..., t_n \doteq u_n$.

Corollaire 2 (Existence d'un unificateur principal).

Tout problème d'unification P soluble a une solution principale. [Preuve page 26]

Corollaire 3 (Unicité de l'unificateur principal).

Si un problème P est soluble avec un unificateur principal θ en forme résolue, alors θ est unique à renommage près. [Preuve page 27]

2.5 Flots et câblages

Faisons des remarques préliminaires sur l'essence de l'élimination des coupures par calcul des chemins persistants. Il y a plusieurs points de vues possibles.

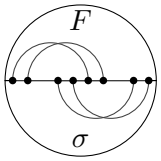


FIGURE 3

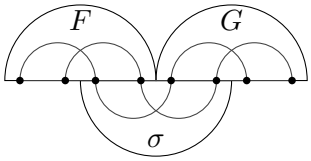


FIGURE 4

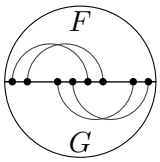


FIGURE 5

— On remarque que les chemins dans un réseau changent de direction lors du passage sur une coupure ou un axiome. De plus, les conclusions des axiomes peuvent être vus comme des sortes de zones d'interactions. Si on représente un axiome par un demi-cercle F et les coupures par un demi-cercle σ (figure 3), on peut voir le calcul des chemins persistants comme le calcul des chemins alternants entre F et σ qui sont maximaux (pour avoir une exécution complète). On peut aussi voir F et σ comme des permutations sur les conclusions des atomes dont on fait le produit.

— On pourrait avoir un point de vue où les coupures relient deux sous-parties disjointes du véhicule qu'on note $F + G$ (figure 4). Une arête de coupure dans σ identifie en fait deux points d'interaction (conclusion des axiomes) de F et G. On peut simplifier ce modèle en connectant directement les points d'interactions de F et G en suivant σ (figure 5). L'élimination des coupures revient donc à calculer l'ensemble des chemins alternants entre F et G. On peut maintenant limiter l'essence calculatoire d'une preuve à son véhicule qu'on connecte à d'autres véhicules.

Les flots nous donnent un modèle permettant, en quelque sorte, de désynchroniser l'élimination des coupures en partant du modèle de la figure 5 en plus de pouvoir être étendu à des fragments plus complexes de la logique linéaire (additif et exponentiel). On va représenter les *points d'interaction* ou *lieux* par des termes ouverts sur une signature $\mathcal{S}$ fixée et leur connectivité par l'unification de termes. Ce sont des sortes d'adresses (similairement aux poids dans L^*) décrivant des positions dans le réseau. L'élimination des coupures sera une sorte de résolution d'adresse.

Definition 19 (Flots).

Un **flot** est une paire ordonnée $t \leftarrow u$ de termes du premier ordre telle que $\mathtt{var}(t) = \mathtt{var}(u)$. On considère les flots égaux à renommage près. Exemples : $x \leftarrow x$ et $f(g(y), y) \leftarrow g(y)$. Le flot $id = x \leftarrow x$ est appelé **flot identité**.

La présence de variables permet de traiter naturellement les coupures et les axiomes sur des formules complexes. Par exemple une coupure entre $A \otimes B$ et $A^\perp \parr B^\perp$ peut avoir des chemins passant par les prémisses gauches ou droites. On représente ces possibilités par une variable en attente de prendre une valeur.

Definition 20 (Symétrisation).

La **symétrisation** d'un flot $t \leftharpoonup u$ est le flot $(t \leftharpoonup u) + (u \leftharpoonup t)$ noté $t \rightleftharpoons u$ ou $u \rightleftharpoons t$.

Definition 21 (Produit).

Le **produit** de deux flots $f = u \leftharpoonup v$ et $g = t \leftharpoonup w$ est défini par

$$(u \leftharpoonup v)(t \leftharpoonup w) := \theta u \leftharpoonup \theta w$$

où f et g sont renommés de sorte à ne pas partager de variable et θ est l'unificateur principal de v et t. S'il n'existe pas de tel unificateur, le produit n'est pas défini et $fg = \bot$.

Exemple. On a le produit suivant : $(f(x) \leftharpoonup x)(g(x) \leftharpoonup x) = (f(x) \leftharpoonup x)(g(y) \leftharpoonup y) = f(g(y)) \leftharpoonup y$. Le flot de droite est renommé puis on applique $\theta = \{x \mapsto g(y)\}$ sur les extrémités formant le flot résultat.

Definition 22 (Câblage de flots).

Un **câblage de flots** ou simplement *câblage* quand il n'y a pas d'ambiguité est un ensemble de flots. On peut étendre le produit de flots à celui de produit de deux câblages F et G par :

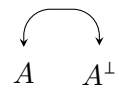
$$FG = \{fg \mid f \in F, g \in G, fg \neq \bot\}$$

Par définition on peut définir la puissance F^n avec $n \in \mathbb{N}$ comme :

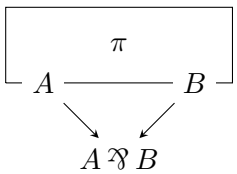
$$F^0 = \{id\} \qquad\qquad F^{n+1} = F^n F$$

Definition 23 (Traduction vers les flots).

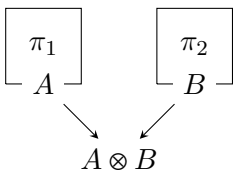
On reconstruit le véhicule des *réseaux de preuve* (et non des structures de preuves) avec des symétrisations de flots où chaque côté représente un lieu.



$$\overset{\|\cdot\|}{\rightsquigarrow} \quad (p_a \cdot x) \rightleftharpoons (p_{a^\perp} \cdot x)$$



$$\overset{\|\cdot\|}{\rightsquigarrow} \quad \|\pi\| \, [p_A(x) := p_{A \parr B} \cdot \mathtt{l} \cdot x, p_B(x) := p_{A \parr B} \cdot \mathtt{r} \cdot x]$$



$$\overset{\|\cdot\|}{\rightsquigarrow} \quad \|\pi_1\| \, [p_A(x) := p_{A \otimes B} \cdot \mathtt{l} \cdot x] + \|\pi_2\| \, [p_B(x) := p_{A \otimes B} \cdot \mathtt{r} \cdot x]$$

L'expression $[t := u]$ met à jour le terme t en le remplaçant par u. On utilise l'opérateur $f + g$ comme raccourci pour $\{f\} \cup \{g\}$. Le symbole $\cdot$ est un symbole binaire de $\mathcal{S}$ associatif à droite c'est-à-dire qu'on a $x \cdot y \cdot z = x \cdot (y \cdot z)$.

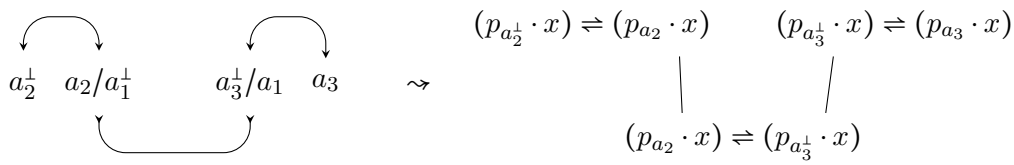
Definition 24 (Exécution).

L'**exécution** $\mathtt{Ex}_f(F,G)$ (figure 5) pour deux câblages F et G est définie par l'ensemble des chaînes maximales de compositions qui alternent entre les flots de F et de G :

$$\mathtt{Ex}_f(F,G) = (1-\sigma^2)(E_{F,F} + E_{F,G} + E_{G,G} + E_{G,F})(1-\sigma^2)$$

où $E_{F,F} = \sum_{k=0}^{\infty} F(GF)^k, E_{F,G} = \sum_{k=0}^{\infty} G(GF)^k, E_{G,G} = \sum_{k=0}^{\infty}(GF)^k G, E_{G,F} = \sum_{k=0}^{\infty} F(GF)^k G$ qui représentent des chemins alternants commençant et finissant par F ou G. Les termes $(1-\sigma^2)$ suppriment les compositions non maximales, c'est-à-dire celles dont le terme source et/ou le terme but appartient à la coupure. La notation est empruntée à la formulation de l'execution dans les algèbres d'opérateurs, où σ est une symétrie partielle représentant la coupure. Dans ce cas, σ^2 est donc une projection sur un sous-espace (le lieu de la coupure, ici l'intersection des lieux de F et G), et $1-\sigma^2$ est une projection sur le sous-espace complémentaire. La composition à droite et à gauche par cette projection a donc cet effet de restreindre le domaine et le codomaine de l'opérateur calculé au sous-espace complémentaire de la coupure, ce qui correspond ici à supprimer les compositions non maximales.

Si on prend le réseau de preuve précédent avec une coupure entre $a_1^\perp \parr a_1$ et $a_2 \otimes a_3^\perp$ interprété comme la connexion de deux véhicules et sa composition associée dans les flots, on obtient :



Il n'y a qu'un chemin maximal alternant produisant le flot $(p_{a_2^\perp} \cdot x) \rightleftharpoons (p_{a_3} \cdot x)$ qui représente bien la forme normale voulue.

3 Modèle stellaire

Girard introduit la syntaxe transcendantale comme successeur de la géométrie de l'interaction avec un espace d'entités formé des *étoiles et constellations* [Gir13a ; Gir13b ; Gir17]. Ce modèle part du constat que les flots sont bidirectionnels $t \rightleftharpoons u$ dans l'interprétation de la logique et ne sont adéquats que pour modéliser la partie calculatoire des preuves (véhicule).

Ce nouveau programme propose de généraliser le modèle de flots pour prendre en considération le gabarit (modélisation des tests) vu comme typage afin d'étendre l'espace des entités logique, avoir des constructions riches de types/formules et reconstituer d'autres phénomènes logiques (entre autre une explication des individus du premier ordre). De plus, par sa portée philosophique, ce programme propose de reconstruire la logique sans référence nécessaire à l'infini afin de donner une explication finitaire de la logique.

3.1 Etoiles et constellations

Definition 25 (Couleur).

Pour une signature $\mathcal{S}$ fixée, on fixe un ensemble de symboles de fonctions unaires appelés **couleurs** utilisés comme préfixe des $\mathcal{S}$-termes.

Exemple. Le terme $\alpha(t)$ est de couleur α. On utilisera la notation $\alpha.t$. Les couleurs existent par paires duales. On notera $\overline{\alpha}$, la couleur duale de α.

Definition 26 (Rayons).

Pour une signature $\mathcal{S}$ fixée, un **rayon** est un $\mathcal{S}$-terme préfixé par une couleur. Si un rayon n'est pas préfixé par une couleur, il est qualifié d'*incolore*. Deux rayons sont **duaux** s'ils sont de couleur duales.

Definition 27 (Étoiles).

Une **étoile** désignée par σ est un multi-ensemble non vide et fini $[\![t_1, ..., t_n]\!]$ de rayons.

- $\texttt{vars}(\sigma)$ comme l'ensemble des variables apparaissant dans σ.
- pour toute substitution θ, on a $\theta\sigma = [\![\theta t_1, ..., \theta t_n]\!]$ comme l'application de θ à tous les rayons de σ.
- $\texttt{colors}(\sigma)$ comme l'ensemble des couleurs qui apparaissent dans σ.
- La coloration de $\sigma = [\![t_1, ..., t_n]\!]$ par α notée $\alpha.\sigma$ représente σ où tous les rayons incolores sont préfixés par la couleur α.

Ce sont des regroupements de termes avec des variables connectées entre elles. Exemples : $[\![\alpha.x, \overline{\alpha}.x, \beta.x]\!]$ et $[\![\alpha.f(x), h(f(x)), \beta.g(x)]\!]$

Definition 28 (α-équivalence).

Deux étoiles σ_1 et σ_2 sont α-**équivalentes** noté $\sigma_1 \approx_\alpha \sigma_2$ quand il existe un renommage α tel que $\sigma_1 = \alpha\sigma_2$ ou $\sigma_2 = \alpha\sigma_1$. Par commodité, on considérera toujours que l'égalité entre étoiles et $\approx_\alpha$ coïncident.

Definition 29 (Constellations).

Une **constellation** est un multi-ensemble d'étoiles $\sigma_1 + ... + \sigma_n$ qu'on désigne par les symboles Γ, Δ.

- Si une constellation possède au moins un rayon coloré alors elle est **implicite** et sinon **explicite**.
- On définit $\texttt{colors}(\sigma_1 + ... + \sigma_n) = \texttt{colors}(\sigma_1) \cup ... \cup \texttt{colors}(\sigma_n)$.
- La coloration de Σ par α avec $\alpha.\Sigma = \alpha.\sigma_1 + ... + \alpha.\sigma_n$.
- Chaque étoile doit avoir un ensemble de variables disjoint des autres (en d'autres termes, les variables sont locales aux étoiles).

3.2 Diagrammes et normalisation

L'évaluation des constellations consiste à former toutes les possibilités de connexions entre les rayons duaux et à les réduire par fusion des étoiles et unification des rayons connectés. Chaque réduction produit une étoile qui fera partie d'une constellation résultat. Cette idée sera décrite formellement par la suite.

Ce modèle a la spécificité d'avoir un mécanisme presque identique à la *méthode de résolution* [Rob+65] utilisée en programmation logique. Une clause $A_1, ..., A_n \vdash B$ peut être traduite en une étoile $[\![\overline{\alpha}.A_1, ..., \overline{\alpha}.A_n, \alpha.B]\!]$ (avec B incolore si c'est un but qui n'est pas destiné à disparaître). L'évaluation que nous considérons est une variante de la méthode de résolution.

Definition 30 (Graphe d'unifiabilité).

Le **graphe d'unifiabilité** $\mathcal{U}(\Sigma)$ d'une constellation Σ est un *multigraphe* où :

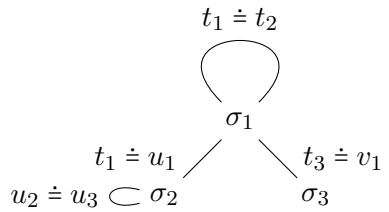
— l'ensemble des sommets est l'ensemble induit par Σ. On les étiquette avec les étoiles correspondantes.

— on a une arête entre deux étoiles pour chacun de leurs rayons duaux unifiables. On étiquette les arêtes par l'équation $t \doteq u$ si elle relie deux rayons $\alpha.t$ et $\overline{\alpha}.u$ pour une couleur α.

Cela représente l'ensemble des connexions potentielles dans une constellation.

Exemple. Si on considère tous les rayons duaux comme étant unifiables, on a :

$$\begin{array}{lcll} \sigma_1 & = & [\![\alpha.t_1, \overline{\alpha}.t_2, \gamma.t_3]\!] & + \\ \sigma_2 & = & [\![\overline{\alpha}.u_1, \beta.u_2, \overline{\beta}.u_3]\!] & + \\ \sigma_3 & = & [\![\overline{\gamma}.v_1]\!] & \end{array} \qquad \text{produit}$$



Definition 31 (Diagramme).

Soit Σ une constellation. Un **diagramme** de base Σ est un arbre D tel qu'il existe un morphisme de multigraphe $\varphi : D \longrightarrow \mathcal{U}(\Sigma)$.

On note $\texttt{links}(D)$ l'ensemble des équations associés aux arêtes de D qu'on appellera **liaisons** et $\texttt{free}(D)$ l'ensemble des **rayons libres** : les rayons de Σ qui ne sont pas dans $\texttt{links}(D)$ qu'on notera `links` et `free` lorsqu'il n'y a pas d'ambiguïtés. Cela représente un schéma décrivant une combinaison d'interaction par rayons entre les étoiles. C'est l'analogue des chemins dans un réseau.

Exemple. On peut avoir les diagrammes suivants pour $\Sigma_1 = [\![\alpha.x, \beta.x]\!] + [\![\overline{\alpha}.f(y), \overline{\beta}.g(y)]\!]$:

$$D_{\Sigma_1} = [\![\alpha.x, \beta.x]\!] + [\![\overline{\alpha}.f(y), \overline{\beta}.g(y)]\!] \qquad \text{et} \qquad D'_{\Sigma_1} = [\![\alpha.x, \beta.x]\!] + [\![\overline{\alpha}.f(y), \overline{\beta}.g(y)]\!]$$

Connecter tous les rayons duaux ne forme pas un arbre et n'est donc pas un diagramme valide. On a $\texttt{links}(D_{\Sigma_1}) = \{x \doteq f(y)\}$, $\texttt{free}(D_{\Sigma_1}) = \{\beta.x, \overline{\beta}.g(y)\}$, $\texttt{links}(D'_{\Sigma_1}) = \{x \doteq g(y)\}$ et $\texttt{free}(D'_{\Sigma_1}) = \{\alpha.x, \overline{\alpha}.f(y)\}$.

Definition 32 (Diagramme saturé).

Soit Σ une constellation. Un **diagramme saturé** de base Σ est un diagramme D de base Σ tel que tout arbre $D' \supset D$, il n'existe pas de morphisme $\varphi' : D' \longrightarrow \mathcal{U}(\Sigma)$ tel que $\varphi'_{|D} = \varphi$ où $\varphi'_{|D}$ est φ' restreint à D.

C'est l'analogue des chemins maximaux dans un réseau. Il faut tout connecter jusqu'à qu'il ne reste plus de connexions possibles.

Exemple. Sur la constellation Σ précédente on a pas de diagramme saturé car on peut toujours utiliser d'autres occurrences pour créer des diagrammes plus grands. Cependant, on peut avoir le diagramme saturé suivant :

$$D_{\Sigma_2} = [\![\alpha.x, \beta.x]\!] + [\![\overline{\alpha}.f(y), y]\!] + [\![\overline{\beta}.f(z), \gamma.g(z), z]\!]$$

Il n'y a pas d'autres liaisons possibles même en ajoutant de nouvelles occurrences d'étoiles.

On évalue un diagramme par la notion *de fusion* qui consiste à fusionner/contracter les étoiles par unification et anihilation des rayons connectés.

Definition 33 (Réduction par fusion).

> Soit D un diagramme de liaisons `links` et de rayons libres `free`. La **réduction par fusion** de D est définie par le choix d'une équation $l \in \texttt{links}$:
>
> $$\sigma_1 + \ldots + \sigma_i + \ldots + \sigma_j + \ldots + \sigma_n \quad \rightsquigarrow_l \quad \sigma_1 + \ldots + (\texttt{Sol}(l)\sigma_i \cup \texttt{Sol}(l)\sigma_j) + \ldots + \sigma_n$$
>
> où Γ est un ensemble de liaisons. Par extension, on peut définir la clôture réflexive transitive $\rightsquigarrow^*$ de $\rightsquigarrow$ par $D \rightsquigarrow_{l_1} \ldots \rightsquigarrow_{l_n} D'$ où $\{l_1, \ldots, l_n\} = \texttt{links}$. La **procédure de réduction par fusion** réduit un diagramme en appliquant une réduction par fusion sur toutes les liaisons.

Exemple. La réduction par fusion sur D_{Σ_2} se déroule ainsi :

$$[\![\alpha.x, \beta.x]\!] + [\![\overline{\alpha}.f(y), y]\!] + [\![\overline{\beta}.f(z), \gamma.g(z), z]\!]$$

$$\rightsquigarrow_{x \doteq f(y)} [\![\beta.f(y), y]\!] + [\![\overline{\beta}.f(z), \gamma.g(z), z]\!] \rightsquigarrow_{f(y) \doteq f(z)} [\![z, \gamma.g(z), z]\!]$$

On peut imaginer une forme alternative de réduction des diagrammes qui fournit une résolution de toutes les équations :

Definition 34 (Actualisation).

> Soit D un diagramme. L'**actualisation** de D est $\Downarrow D := \texttt{Sol}(\texttt{links})(\texttt{free})$. On résout les équations sous-jacentes et on applique l'unificateur résultant sur l'ensemble des rayons libres (non connectés).

Exemple. La constellation D_{Σ_2} produit le problème $P = \{x \doteq f(y), x \doteq f(z)\}$ avec $\texttt{free}(D_{\Sigma_2}) = [\![y, \gamma.g(z), z]\!]$. La résolution de P donne $\theta = \{x \mapsto f(y), y \mapsto z\}$ et on obtient l'actualisation $\theta\texttt{free} = [\![z, \gamma.g(z), z]\!]$.

Ces deux procédures de réduction sont équivalentes. On travaillera avec l'actualisation par simplicité.

Théorème 4 (Équivalence entre actualisation et réduction par fusion).

> Pour tout diagramme correct D, on a $D \rightsquigarrow^* \Downarrow D$. [Preuve page 29]

Definition 35 (Diagramme correct et incorrect).

> L'actualisation d'un diagramme peut échouer ou non selon la présence de conflits d'unification.
>
> — si $\Downarrow D$ n'est pas défini (une unification échoue durant l'actualisation) alors D est qualifié de diagramme **incorrect**.
>
> — sinon $\Downarrow D$ produit une étoile appelée **étoile résiduelle** et D est qualifié de diagramme **correct**.

Exemple. Pour la constellation Σ_1 définie plus haut, les diagrammes à 1 liaison sont corrects mais l'unique diagramme de 2 liaisons est incorrect.

On cherche maintenant à évaluer une constellation entière en calculant tous les diagrammes saturés possibles chacun produisant une étoile faisant partie d'une constellation résultat.

Definition 36 (Normalisation).

La **normalisation** ou l'**exécution** de Σ est définie par :

$$\mathtt{Ex}^\star(\Sigma) := \mathtt{normal}\big(\Downarrow \{D \mid D \text{ est correct de base } \Sigma \text{ et saturé}\}\big)$$

où $\mathtt{normal}(\Sigma)$ supprime les rayons incolores et renomme les étoiles de sorte à avoir une constellation. Ce choix est arbitraire et n'est pas investigué dans ce rapport.

Definition 37 (Propriété de normalisation forte).

La propriété de **normalisation forte** garantissant la terminaison de la normalisation et la production d'un résultat est donnée par les conditions suivantes : (1) Le nombre de diagrammes corrects doit être fini (2) Aucun diagramme clos sans rayons libre n'est correct (pour éviter la production de l'étoile vide).

Comme les diagrammes de tailles n sont inclus dans les diagrammes de taille $n+1$, la première condition est équivalente à dire qu'il existe un k tel que tous les diagrammes de taille $n > k$ sont incorrects. Une constellation qui satisfait cette propriété est qualifée de **fortement normalisable**.

Théorème 5 (Confluence de la normalisation).

Les deux procédures de normalisation suivantes sont équivalentes :

— On normalise les couleurs une par une

— On normalise toutes les couleurs.

[Preuve page 30]

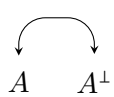
Ce résultat nous assure qu'on a un espace de calcul cohérent où l'ordre d'évaluation n'a pas de conséquence sur le résultat. C'est point majeur qui nous assurera aussi l'associativité de l'élimination des coupures permettant l'existence de modèles catégoriques.

3.3 Véhicule et Gabarit

La syntaxe transcendantale prend comme véhicule le contenu calculatoire des preuves (axiomes et coupures) mais exploite plus spécialement le gabarit en le voyant comme typage du véhicule, ce qui n'était pas pris en compte dans le modèle de flot.

Definition 38 (Véhicule sans coupure).

On traduit le véhicule d'un *réseau de preuve* (et non une structure de preuve) de MLL π par induction sur π :



$$\overset{\|\cdot\|}{\rightsquigarrow} \quad [\![p_A(x), p_{A^\perp}(x)]\!]$$

$$\begin{tikzcd} \boxed{\begin{array}{c} \pi' \\ A \quad\text{———}\quad B \end{array}} \arrow[dr] & & \\ & A \parr B & \end{tikzcd} \quad \overset{\|\cdot\|}{\rightsquigarrow} \quad \|\pi'\| \, [p_A(x) := p_{A \parr B}(l \cdot x), p_B(x) := p_{A \parr B}(r \cdot x)]$$

$$\begin{tikzcd} \boxed{\begin{array}{c} \pi_1 \\ A \end{array}} \arrow[dr] & & \boxed{\begin{array}{c} \pi_2 \\ B \end{array}} \arrow[dl] \\ & A \otimes B & \end{tikzcd} \quad \overset{\|\cdot\|}{\rightsquigarrow} \quad \|\pi_1\| \, [p_A(x) := p_{A \otimes B}(l \cdot x)] + \|\pi_2\| \, [p_B(x) := p_{A \otimes B}(r \cdot x)]$$

où $[t := u]$ met à jour le terme t en le remplaçant par u. La traduction donne une localisation des formules des axiomes par une adresse représentée par un terme.

On montre en annexe que les étoiles et les constellations forment un modèle de MLL en se basant sur la notion de modèle de la logique linéaire par biorthogonalité menant à une catégorie $*$-autonome.

Definition 39 (Test).

Un **test** $\|t\|$ d'une structure de preuve π de MLL est défini par une traduction des sommets d'un test $t \in G(\pi)$:

$$A, A^\perp : [\![\frac{\overline{\alpha}.\mathtt{loc}(A)}{\beta.q_A(x)}]\!] \quad \text{où } A, A^\perp \text{ font partie d'un axiome du réseau}$$

$$A \otimes B : [\![\frac{\overline{\beta}.q_A(x) \quad \overline{\beta}.q_B(x)}{\beta.q_{A \otimes B}(x)}]\!] \quad \text{On fait un usage essentiel des étoiles : on a 3 termes liés.}$$

$$A \parr_L B : [\![\frac{\overline{\beta}.q_A(x)}{\beta.q_{A \parr B}(x)}]\!] + [\![\frac{\overline{\beta}.q_A(x)}{}]\!] \qquad A \parr_R B : [\![\frac{\overline{\beta}.q_A(x)}{\beta.q_{A \parr B}(x)}]\!] + [\![\frac{\overline{\beta}.q_B(x)}{}]\!]$$

$$\text{Coupure} : [\![\overline{\beta}.q_A(x), \overline{\beta}.q_{A^\perp}(x)]\!] \qquad \text{Conclusion} : [\![\frac{\overline{\beta}.q_A(x)}{p_A(x)}]\!] \text{ où } A \text{ est une conclusion}$$

Le **gabarit** est l'ensemble de traductions des tests $t \in G(\pi)$.

— Les termes $p_A(x)$ sont des adresses dans le réseau. On introduit $q_A(x) := p_A(\mathsf{g} \cdot x)$ pour rendre le véhicule et le gabarit spatialement distincts.

— Les couleurs permettent de simuler une sorte de flux entrée-sortie. On peut représenter visuellement une étoile comme des sortes de briques que l'on empile, similairement à des règles d'inférence.

Les constellations permettent de placer de nombreuses entités logiques dans le même espace. Il existe une dualité entre les constellations que l'on définit par la convergence de leur interaction :

Definition 40 (Orthogonalité).

Soient Σ_1 et Σ_2 deux constellations. On a $\Sigma_1 \perp \Sigma_2$ si $\mathtt{Ex}^\star(\Sigma_1 \cup \Sigma_2)$ a la propriété de normalisation forte. On définit l'**orthogonal** d'un ensemble de constellations par : $A^\perp = \{\Sigma \mid \forall \Sigma' \in A. \Sigma \perp \Sigma'\}$.

On peut aussi reconstruire la notion de formule logique ou type en restant dans le même espace (étoiles et constellations) :

Definition 41 (Type).

Un **type** est un ensemble de constellations $\mathbf{A}$ tel que il existe un ensemble de constellations B tel que $\mathbf{A} = B^\perp$.

C'est-à-dire qu'un ensemble de constellations est un type s'il a toujours un ensemble dual avec qui il peut interagir correctement. Cela fait apparaître l'idée de *test d'usage* : une constellation fait partie d'un type si elle passe tous les tests de convergence par interaction avec les constellations du type dual. Autrement dit, un type est constitué d'entités testables par un type dual. On peut avoir une définition alternative par une double orthogonalité (biorthogonal dans la littérature) qui explicite l'idée d'un espace "clos par dualité".

Théorème 6 (Clôture par biorthogonal).

Soit $\mathbf{A}$ un ensemble de constellation. On a $\mathbf{A} = \mathbf{A}^{\perp\perp}$ si et seulement si $\mathbf{A}$ est un type. [Preuve page 34]

Théorème 7 (Modèle dénotationnel de MLL).

Le modèle stellaire forme un modèle dénotationnel de MLL. [Esquisse page **??**]

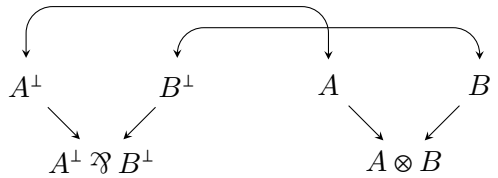
3.4 Usine

On reconstruit le critère de correction de Danos-Regnier grâce aux étoiles et constellations.

Definition 42 (Critère de correction stellaire).

Soit π une structure de preuve de MLL de conclusion $\Gamma = \{A_1, ..., A_n\}$. Pour tout test $t \in G(\pi), \mathtt{Ex}^\star(\alpha. \|V(\pi)\| \cup \|t\|) = [\![p_\Gamma := p_{A_1}(x_1), ..., p_{A_n}(x_n)]\!]$ pour certains $x_1, ..., x_n$.

Illustrons avec le réseau de preuve suivant et sa traduction associée avec le test $\parr_L$:

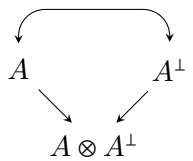


$$\alpha.\mathcal{V} = \begin{array}{l} [\![\alpha.p_{A\otimes B}(l\cdot x), \alpha.p_{A^\perp \parr B^\perp}(l\cdot x)]\!] + \\ [\![\alpha.p_{A\otimes B}(r\cdot x), \alpha.p_{A^\perp \parr B^\perp}(r\cdot x)]\!] \end{array}$$

$$\mathcal{T}_L = \begin{array}{l} [\![\frac{\overline{\alpha}.p_{A^\perp \parr B^\perp}(\mathtt{l}\cdot x)}{\beta.q_{A^\perp}(x)}]\!] + [\![\frac{\overline{\alpha}.p_{A^\perp \parr B^\perp}(\mathtt{r}\cdot x)}{\beta.q_{B^\perp}(x)}]\!] + [\![\frac{\overline{\alpha}.p_{A\otimes B}(x)}{\beta.q_A(\mathtt{l}\cdot x)}]\!] + [\![\frac{\overline{\alpha}.p_{A\otimes B}(x)}{\beta.q_B(\mathtt{r}\cdot x)}]\!] + \quad \text{(ax)} \quad [\![\frac{\overline{\beta}.q_A(x), \overline{\beta}.q_B(x)}{\beta.q_{A\otimes B}(x)}]\!] + \quad (\otimes) \\ [\![\frac{\overline{\beta}.q_{A^\perp}(x)}{\beta.q_{A^\perp \parr B^\perp}(x)}]\!] + [\![\frac{\overline{\beta}.q_{A^\perp}(x)}{}]\!] + \quad (\parr_L) \quad [\![\frac{\overline{\beta}.q_{A^\perp \parr B^\perp}(x)}{p_{A^\perp \parr B^\perp}(x)}]\!] + [\![\frac{\overline{\beta}.q_{A\otimes B}(x)}{p_{A\otimes B}(x)}]\!] \quad \text{(conclusions)} \end{array}$$

On remarque que si on connecte toutes les étoiles de façon évidente, toutes les couleurs duales s'annulent pour ne laisser que les conclusions. On peut aussi essayer avec l'autre test ($\parr_R$). Les graphes d'unifiabilités produits étant acycliques (grâce à l'étoile $\parr_L$), le nombre de diagrammes est fini (donc le nombre de diagrammes corrects aussi).

Cependant, si on veut simuler une structure qui ne satisfait pas le critère de correction, il faut libéraliser la définition de véhicule (on l'omet ici). Un tenseur sur les conclusions d'un axiome ne relie pas deux réseaux disjoints. On devrait donc avoir le véhicule $[\![p_{A\otimes A^\perp}(l\cdot x), \alpha.p_{A\otimes A^\perp}(r\cdot x)]\!]$.



$$\alpha.\mathcal{V} = \llbracket \alpha.p_{A\otimes A^\perp}(l \cdot x), \alpha.p_{A\otimes A^\perp}(r \cdot x) \rrbracket$$
$$\mathcal{T} = \llbracket \frac{\overline{\alpha}.p_{A\otimes A^\perp}(\mathtt{l}\cdot x)}{\beta.q_A(x)} \rrbracket + \llbracket \frac{\overline{\alpha}.p_{A\otimes A^\perp}(\mathtt{r}\cdot x)}{\beta.q_{A^\perp}(x)} \rrbracket + \llbracket \frac{\overline{\beta}.q_A(x), \overline{\beta}.q_{A^\perp}(x)}{\beta.q_{A\otimes A^\perp}(x)} \rrbracket + \llbracket \frac{\overline{\beta}.q_{A\otimes A^\perp}(x)}{p_{A\otimes A^\perp}(x)} \rrbracket$$

On a un cycle entre l'unique étoile du véhicule $\llbracket \alpha.p_{A\otimes A^\perp}(l \cdot x), \alpha.p_{A\otimes A^\perp}(r \cdot x) \rrbracket$ et l'étoile du tenseur $\llbracket \frac{\overline{\beta}.q_A(x), \overline{\beta}.q_{A^\perp}(x)}{\beta.q_{A\otimes A^\perp}(x)} \rrbracket$. Comme chaque liaison est de la forme $t \doteq t$ alors on peut créer une infinité de diagrammes saturés en ajoutant des occurrences d'étoiles en nombre arbitraire.

Corollaire 8 (Simulation de correction).

Soit π une structure de preuve de MLL. On a π est correct si et seulement si π satisfait le critère de correction stellaire. [Preuve page 36]

3.5 Usage

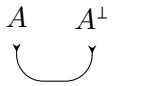
On étend la traduction du véhicule pour prendre en compte l'idée de coupure entre deux véhicules.

Definition 43 (Coupure).

Une **coupure** est l'usage d'une règle de la forme $\dfrac{\vdash \Gamma, A \otimes A^\perp}{\vdash \Gamma, [A \otimes A^\perp]}$. C'est une variante de la règle originale $\dfrac{\vdash \Gamma, A \quad \vdash \Delta, A^\perp}{\vdash \Gamma, \Delta}$ où $[A]$ est une formule destinée à être effacée.

Cette formulation de la coupure nous permet d'avoir une traduction naturelle dans les constellations où on se repose sur la traduction du tenseur et on ajoute une étoile de "feedback" $\llbracket \frac{\overline{\beta}.p_A(x), \overline{\beta}.p_{A^\perp}(x)}{} \rrbracket$ permettant d'annuler les formules concernées par la coupure.

Definition 44 (Traduction de la coupure pour le véhicule).



$$\overset{\|\cdot\|}{\rightsquigarrow} \quad \|\pi\| \, [p_A(x) := \beta.p_A(x), p_{A^\perp}(x) := \beta.p_{A^\perp}(x)] + \llbracket \frac{\overline{\beta}.p_A(x), \overline{\beta}.p_{A^\perp}(x)}{} \rrbracket$$

On peut noter que l'étoile de feedback est aussi présente dans la traduction des sommets coupures pour le gabarit.

Théorème 9 (Simulation de l'élimination des coupures).

Soit π un réseau de preuve de MLL. Si π se réduit en π' par élimination des coupures alors $\mathtt{Ex}^\star(\|\pi\|) = \|\pi'\|$.

Preuve.

L'idée de la preuve est esquissée par Girard [Gir17].

□

Nous avons obtenu une reconstruction des preuves de MLL par la juxtaposition d'un véhicule et d'un gabarit $\mathcal{V}+\mathcal{G}$ où chacun peut posséder des coupures (et donc possède une notion d'évaluation). Le véhicule représente la partie calculatoire de la preuve et le gabarit la cohérence (typage) du véhicule.

Références

[AB14] Clément Aubert et Marc Bagnol. “Unification and logarithmic space”. In : *Rewriting and Typed Lambda Calculi*. Springer, 2014, p. 77-92.

[ABS16] Clément Aubert, Marc Bagnol et Thomas Seiller. “Unary resolution : Characterizing ptime”. In : *International Conference on Foundations of Software Science and Computation Structures*. Springer. 2016, p. 373-389.

[Bag14] Marc Bagnol. “On the resolution semiring”. Thèse de doct. Aix-Marseille Université, 2014.

[BP01] Patrick Baillot et Marco Pedicini. “Elementary complexity and geometry of interaction”. In : *Fundamenta Informaticae* 45.1-2 (2001), p. 1-31.

[Cur34] Haskell B Curry. “Functionality in combinatory logic”. In : *Proceedings of the National Academy of Sciences of the United States of America* 20.11 (1934), p. 584.

[Dan90] Vincent Danos. “La Logique Linéaire appliquée à l’étude de divers processus de normalisation (principalement du Lambda-calcul)”. Thèse de doct. Paris 7, 1990.

[DR89] Vincent Danos et Laurent Regnier. “The structure of multiplicatives”. In : *Archive for Mathematical logic* 28.3 (1989), p. 181-203.

[DR95] Vincent Danos et Laurent Regnier. “Proof-nets and the Hilbert space”. In : *London Mathematical Society Lecture Note Series* (1995), p. 307-328.

[Gen35a] Gerhard Gentzen. “Untersuchungen über das logische Schließen. I”. In : *Mathematische zeitschrift* 39.1 (1935), p. 176-210.

[Gen35b] Gerhard Gentzen. “Untersuchungen über das logische Schließen. II”. In : *Mathematische Zeitschrift* 39.1 (1935), p. 405-431.

[Gir01] Jean-Yves Girard. “Locus solum : From the rules of logic to the logic of rules”. In : *Mathematical structures in computer science* 11.3 (2001), p. 301-506.

[Gir06] Jean-Yves Girard. “Geometry of interaction IV : the feedback equation”. In : *Logic Colloquium*. T. 3. 2006, p. 76-117.

[Gir11] Jean-Yves Girard. “Geometry of interaction V : logic in the hyperfinite factor”. In : *Theoretical Computer Science* 412.20 (2011), p. 1860-1883.

[Gir13a] Jean-Yves Girard. “Geometry of interaction VI : a blueprint for transcendental syntax”. In : *preprint* (2013).

[Gir13b] Jean-Yves Girard. “Three lightings of logic (Invited Talk)”. In : *Computer Science Logic 2013 (CSL 2013)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. 2013.

[Gir16a] Jean-Yves Girard. *Le fantôme de la transparence*. Éditions Allia, 2016.

[Gir16b] Jean-Yves Girard. “Transcendental syntax II : Non-deterministic case”. In : *Logical Methods in Computer Science, to appear* (2016).

[Gir16c] Jean-Yves Girard. “Transcendental syntax III : equality”. In : *Logical Methods in Computer Science* (2016).

[Gir17] Jean-Yves Girard. “Transcendental syntax I : deterministic case”. In : *Mathematical Structures in Computer Science* 27.5 (2017), p. 827-849.

[Gir87a] Jean-Yves Girard. “Linear logic”. In : *Theoretical computer science* 50.1 (1987), p. 1-101.

[Gir87b] Jean-Yves Girard. “Multiplicatives”. In : (1987).

[Gir88] Jean-Yves Girard. “Geometry of interaction 2 : Deadlock-free algorithms”. In : *International Conference on Computer Logic*. Springer. 1988, p. 76-93.

[Gir89a] Jean-Yves Girard. “Geometry of interaction 1 : Interpretation of System F”. In : *Studies in Logic and the Foundations of Mathematics*. T. 127. Elsevier, 1989, p. 221-260.

[Gir89b] Jean-Yves Girard. “Towards a geometry of interaction”. In : *Contemporary Mathematics* 92.69-108 (1989), p. 6.

[Gir95] Jean-Yves Girard. “Geometry of interaction III : accommodating the additives”. In : *London Mathematical Society Lecture Note Series* (1995), p. 329-389.

[Her30] Jacques Herbrand. “Recherches sur la théorie de la démonstration”. Thèse de doct. Université de Paris, 1930.

[How80] William A Howard. “The formulae-as-types notion of construction”. In : *To HB Curry : essays on combinatory logic, lambda calculus and formalism* 44 (1980), p. 479-490.

[HS03] Martin Hyland et Andrea Schalk. “Glueing and orthogonality for models of linear logic”. In : *Theoretical computer science* 294.1-2 (2003), p. 183-231.

[Maz15] Damiano Mazza. “Simple parsimonious types and logarithmic space”. In : *24th EACSL Annual Conference on Computer Science Logic (CSL 2015)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. 2015.

[Maz17] Damiano Mazza. “Polyadic Approximations in Logic and Computation”. In : (2017).

[MM82] Alberto Martelli et Ugo Montanari. “An efficient unification algorithm”. In : *ACM Transactions on Programming Languages and Systems (TOPLAS)* 4.2 (1982), p. 258-282.

[MT15] Damiano Mazza et Kazushige Terui. “Parsimonious types and non-uniform computation”. In : *International Colloquium on Automata, Languages, and Programming*. Springer. 2015, p. 350-361.

[Reg92] Laurent Regnier. “Lambda-calcul et réseaux”. Thèse de doct. Paris 7, 1992.

[Rob+65] John Alan Robinson et al. “A machine-oriented logic based on the resolution principle”. In : *Journal of the ACM* 12.1 (1965), p. 23-41.

[Sei12a] Thomas Seiller. “Interaction graphs : multiplicatives”. In : *Annals of Pure and Applied Logic* 163.12 (2012), p. 1808-1837.

[Sei12b] Thomas Seiller. “Logique dans le facteur hyperfini : géometrie de l’interaction et complexité”. Thèse de doct. Aix-Marseille Université, 2012.

[Sei13] Thomas Seiller. “Interaction graphs : Exponentials”. In : *arXiv preprint arXiv :1312.1094* (2013).

[Sei17] Thomas Seiller. “Interaction graphs : Graphings”. In : *Annals of Pure and Applied Logic* 168.2 (2017), p. 278-320.

[Sei18] Thomas Seiller. “Interaction Graphs : Non-Deterministic Automata”. In : *ACM Transactions on Computational Logic (TOCL)* 19.3 (2018), p. 21.

[Wit09] Ludwig Wittgenstein. *Philosophical investigations*. John Wiley & Sons, 2009.

Annexes

Cette partie contient des informations qui ne sont pas strictement nécessaires à la lecture du document. On y inclut entre autre des détails de preuves de lemmes et théorèmes ainsi que des réflexions conceptuelles afin de prendre du recul sur le travail effectué.

Annexe 1 - Théorie de l'unification

On définit une signature $\mathcal{S} = \texttt{Vars} \uplus \texttt{Func}$ où
- $\texttt{Vars} = \{w, x, y, z, ...\}$ est un ensemble de symboles de variables.
- $\texttt{Func} = \{f^{(n_1)}, g^{(n_2)}, h^{(n_3)}, ...\}$ est un ensemble de symboles de fonctions accompagnés d'une arité.

Tous ces ensembles peuvent être considérés finis puisqu'on peut se restreindre aux symboles dont on a besoin pour un contexte donné. Les $\mathcal{S}$-termes (ou simplement *termes* en l'absence d'ambiguïté) sont générés par la grammaire suivante :

$$t, u, v, w \quad := x \quad | \quad f(t_1, ..., t_n) \qquad\qquad (\mathcal{S}\text{-termes})$$

où $x \in \texttt{Vars}$ et $f^{(n)} \in \texttt{Func}$ pour un certain $n \in \mathbb{N}$. On notera $f^k(t)$ pour $k \in \mathbb{N}$ applications de f à t. On note $\texttt{vars}(t)$ l'ensemble des variables d'un $\mathcal{S}$-terme t et un terme t est clos si $\texttt{vars}(t) = \varnothing$. Exemples : $x, f(x), h(f(x), z)$. Non-exemples : $f(g), x(f), y(f(x))$.

Definition 45 (Substitution).

Une **substitution** est une fonction $\theta = \{\!|x_1 \mapsto t_1, ..., x_n \mapsto t_n|\!\}$ associant une valeur aux variables et définie par son action sur les termes. Si on définit les ensembles suivants pour θ :
- $\texttt{dom}(\theta) = \{x_1, ..., x_n\}$
- $\texttt{img}(\theta) = \{t_1, ..., t_n\}$

alors on peut définir l'action d'une substitution par :

$$\theta(x_i) = t_i \qquad\qquad \theta(y)_{y \in \texttt{Vars} \setminus dom(\theta)} = y \qquad\qquad \theta(f(u_1, ..., u_k)) = f(\theta u_1, ..., \theta u_k)$$

Definition 46 (Composition de substitutions).

La **composition** de deux substitutions θ_1 et θ_2 est définie par $(\theta_1 \circ \theta_2)t = \theta_1(\theta_2 t)$. On la considère associative à droite par défaut c'est-à-dire que $\theta_1 \circ \theta_2 \circ \theta_3$ est une notation pour $\theta_1 \circ (\theta_2 \circ \theta_3)$.

Definition 47 (Unification).

Deux termes t et u sont **unifiables** quand il existe une substitution θ telle que $\theta t = \theta u$. Une telle substitution est aussi appelée **unificateur** de t et u.

Definition 48 (Renommage).

Un **renommage** α est une permutation sur l'ensemble des variables qu'on représente sous la forme d'une substitution α tel que $\texttt{dom}(\alpha) = \texttt{img}(\alpha) = \texttt{Vars}$. Pour tout terme t et renommage α, αt est appelé **renommage** de t.

Definition 49 (Equivalence par renommage).

Deux substitutions θ_1, θ_2 sont α-**équivalents** noté $\theta_1 \approx_\alpha \theta_2$ si et seulement si $\theta_1 = \alpha\theta_2$ ou $\theta_2 = \alpha\theta_1$ pour un certain renommage α.

Comme il est toujours possible de renommer, on considère les substitutions à renommage près et on considère que $=$ est confondu avec $\approx_\alpha$.

Definition 50 (Renommage exclusif).

Des **renommage exclusif** α_1, α_2 pour deux termes t, u, sont des renommages tels que $\texttt{vars}(\alpha_1 t) \cap \texttt{vars}(\alpha_2 u) = \varnothing$.

Definition 51 (Conciliation).

Deux termes t et u sont **conciliables** s'ils sont unifiables après renommage par deux renommages exclusifs. Un tel unificateur est appelé **conciliateur**.

Par exemple, $f(x)$ et x sont concialiables avec $\alpha_1 = id$ et $\alpha_2 = \{x \mapsto y\}$.

Definition 52 (Unificateur principal).

La composition induit un pré-ordre sur les substitutions : $\theta \preceq \psi \iff \exists \theta'.\psi = \theta' \circ \theta$.

Un **unificateur principal** θ pour deux termes t et u est un unificateur de t et u minimal pour l'ordre défini ci-dessus. On note $\theta \sim \psi$ si et seulement si $\theta \preceq \psi$ et $\psi \preceq \theta$.

Definition 53 (Problème d'unification).

Un **problème d'unification** est un ensemble $P = \{t_1 \doteq u_1, ..., t_n \doteq u_n\}$ de paires de termes. Le problème P est **soluble** s'il existe un unificateur θ appelé **solution** de P tel que $\texttt{dom}(\theta) \subseteq \texttt{vars}(P)$ qui est un unificateur pour toutes les équations de P. La solution est **principale** si c'est un unificateur principal pour chacune des équations de P.

Definition 54 (Forme résolue).

Un problème P est en **forme résolue** s'il est de la forme $\{x_1 \doteq t_1, ..., x_n \doteq t_n\}$ où tous les x_i sont des variables distinctes qui n'apparaissent dans aucun t_j. Sa **substitution associée** est définie par $\vec{P} = \{\!|x_1 \mapsto t_1, ..., x_n \mapsto t_n|\!\}$.

Théorème 10 (Idempotence de la forme résolue).

Si θ est en forme résolue alors $\theta \circ \theta = \theta$.

Preuve.

Par définition, $\theta = \{x_1 \mapsto t_1, ..., x_n \mapsto t_n\}$ tel que tous les x_i sont distincts et n'apparaissent dans aucun t_j. Donc une application de θ sur un terme t fait disparaître toutes les occurrences de $x_1, ..., x_n$. Une seconde application de θ n'a donc aucun effet.

□

Definition 55 (Algorithme d'unification).

On se donne les règles suivantes correspondant à l'algorithme d'unification de *Martelli* et *Montanari* [MM82] :

effacer	$P \cup \{t \doteq t\}$	$\rightarrow$	P
ouvrir	$P \cup \{f(t_1, ..., t_n) \doteq f(u_1, ..., u_n)\}$	$\rightarrow$	$P \cup \{t_1 \doteq u_1, ..., t_n \doteq u_n\}$
diriger	$P \cup \{t \doteq x\}$ avec $t \notin \texttt{Vars}$	$\rightarrow$	$P \cup \{x \doteq t\}$
remplacer	$P \cup \{x \doteq t\}$ avec $x \in \texttt{vars}(P)$ et $x \notin \texttt{fv}(t)$	$\rightarrow$	$\{\!|x \mapsto t|\!\}P \cup \{x \doteq t\}$

où $\theta\{x_1 \doteq t_1, ..., x_n \doteq t_n\} = \{\theta x_1 \doteq \theta t_1, ..., \theta x_n \doteq \theta t_n\}$.

L'algorithme appliqué à un problème P transforme successivement P par application des règles ci-dessus :

— Si on atteint une erreur (pas de règle applicable) on renvoie $\perp$.

— Sinon, on a atteint un problème final qu'on renvoie.

On notera $\rightarrow^*$ la clôture réflexive transitive de $\rightarrow$.

On peut caractériser les erreurs par les cas suivants :

— Si on a un conflit de symboles de fonction c'est à dire $f(t_1, ..., t_n) \doteq g(u_1, ..., u_m)$ avec $f \neq g$.

— Si on a une équation circulaire de la forme $x \doteq t$ où $x \in \texttt{fv}(t)$. L'unification est impossible et mène donc vers un échec.

On peut aussi remarquer que la condition $x \in \texttt{Vars}(P)$ permet de "fixer" les substitutions qui sont terminales c'est-à-dire qu'on ne traitera plus.

Théorème 11 (Correction).

Si l'algorithme sur P termine avec une erreur, alors P n'a pas d'unificateur. Si l'algorithme termine avec succès alors on obtient un problème S équivalent en forme résolue.

Preuve.

La preuve se trouve dans [MM82] et procède par analyse de chaque règle. Ce théorème permet d'extraire un unificateur $\vec{S}$ à partir du problème final résultant de l'algorithme d'unification. On peut considérer par commodité que cet algorithme renvoie $\vec{S}$.

□

Corollaire 12 (Existence d'un unificateur principal).

Tout problème d'unification P soluble a une solution principale.

Preuve.

L'algorithme d'unification permet de calculer un unificateur $\theta = \{x_1 \mapsto t_1, ..., x_n \mapsto t_n\}$ en forme résolue pour un problème P. Par la correction de l'algorithme, on en déduit l'existence d'un unificateur pour P. Comme il est en forme résolue alors on a $\theta(x_i) = t_i$ et $\theta(t_i) = t_i$. Soit ψ une solution arbitraire du problème P. On a nécessairement $\psi(x_i) = \psi(t_i)$ (pour rester cohérent avec θ). Donc on a $\psi(x_i) = \psi(t_i) = (\psi \circ \theta)(x_i)$ (par forme résolue) impliquant $\psi = \psi \circ \theta$. Par conséquent, $\theta \leq \psi$ et θ est bien principal.

□

Théorème 13 (Équivalence par renommage).

Soient θ_1 et θ_2 deux substitutions. Si $\theta_1 \sim \theta_2$ alors $\theta_1 \approx_\alpha \theta_2$.

Preuve.

Supposons que $\theta_1 \sim \theta_2$. Par définition, on a donc $\theta_1 \leq \theta_2$ et $\theta_2 \leq \theta_1$. En particulier, il existe ψ tel que $\theta_1 = \psi \circ \theta_2$. On raisonne sur l'action de ψ sur une variable x.

— Supposons que $\psi(x) = f(z_1, ..., z_n)$ avec f un symbole d'arité n quelconque. On a donc altéré une certaine substitution $y \mapsto u \in \theta_2$ qui nous donne $(\psi \circ \theta_2)(y) = \{\!|x \mapsto f(z)|\!\}u = \theta_1(y)$ (par hypothèse). Mais cela contredit $\theta_1 \leq \theta_2$ car $\theta_2(y) = u$ et $\theta_1(y) = \{\!|x \mapsto f(z)|\!\}u$. Il est impossible de trouver un ψ' tel que $\theta_2 = \psi' \circ \theta_1$.

— Même avec argument pour $\psi(x) = f(t_1, ..., t_n)$.

Donc, toutes les variables x doivent forcément être renvoyées sur une autre variable y. La sub-

stitution ψ est donc un renommage.

□

Corollaire 14 (Unicité de l'unificateur principal).

Si un problème P est soluble avec un unificateur principal θ en forme résolue, alors θ est unique à renommage près.

Preuve.

Soient θ_1, θ_2 deux unificateurs principaux pour P. Comme ils sont tous deux principaux on a $\theta_1 \sim \theta_2$. Par le théorème d'équivalence par renommage, $\theta_1 \approx_\alpha \theta_2$.

□

Théorème 15 (Terminaison).

L'algorithme d'unification se termine.

Preuve.

La preuve est décrite dans [MM82] et se base sur la bonne-fondation de $\mathbb{N}^3$ en montrant la décroissance d'une mesure (n_1, n_2, n_3) sur un problème d'unification au fil de l'exécution de l'algorithme où :

- n_1 est le nombre de variables qui apparaissent plusieurs fois à gauche d'une équation.
- n_2 est le nombre total de symboles dans le problème.
- n_3 est la somme du nombre d'équations de la forme $x \doteq x$ et $t \doteq x$ où $x \in \texttt{Vars}$ et $t \notin \texttt{Vars}$.

□

Remarque 16 (Confluence de l'algorithme d'unification).

Deux ordres d'exécutions sans échec de l'algorithme d'unification sur un problème P induisent la même solution à renommage près.

Preuve.

Supposons deux exécution de l'algorithme d'unification sur le problème P produisant les solutions principales $\vec{S}, \vec{S'}$. Comme les solutions principales sont équivalentes à renommage près pour un même problème (par unicité de l'unificateur principal), $\vec{S} \approx_\alpha \vec{S'}$.

□

Definition 56 (Solution induite).

On appelle $\texttt{Sol}(P)$ la **solution induite** par un problème P unique à renommage près produite après application de l'algorithme d'unification. Si l'algorithme termine sur un problème S alors $\texttt{Sol}(P) = \vec{S}$, sinon $\texttt{Sol}(P) = \bot$.

Lemme 17 (Décomposition de solution).

Pour tous problèmes P_1, P_2, on a $\texttt{Sol}(P_1 \cup P_2) = \texttt{Sol}(\texttt{Sol}(P_1)P_2) \circ \texttt{Sol}(P_1)$

Preuve.

Dans cette preuve on considère implicite le renommage de deux termes de sorte à ce qu'ils soient égaux lorsqu'ils sont équivalents à renommage près. Comme l'ordre d'exécution de l'algorithme d'unification est arbitraire, on peut résoudre $P_1 \cup P_2$ par une stratégie particulière en se concentrant sur les équations de P_1 permettant de construire $\texttt{Sol}(P_1)$ (dont le domaine reste fixé car la règle "remplacer" efface les occurrences des variables dans le reste du problème). Il reste $\texttt{Sol}(P_1)P_2$ que l'on résoud pour obtenir $\texttt{Sol}(\texttt{Sol}(P_1)P_2)$. Par correction, même après altération du codomaine pendant cette dernière résolution, on garde une solution de P_1 équivalente à renommage près à $\texttt{Sol}(P_1)$.

Comme $\texttt{Sol}(\texttt{Sol}(P_1)P_2) \circ \texttt{Sol}(P_1)$ est une exécution complète de l'algorithme d'unification sur $P_1 \cup P_2$ on obtient $\texttt{Sol}(P_1 \cup P_2)$.

□

Annexe 2 - Théorème de confluence

On écrit $D = (\texttt{links}, \texttt{free})$ comme raccourci pour un diagramme D tel que $\texttt{links}(D) = \texttt{links}$ et $\texttt{free}(D) = \texttt{free}$.

Lemme 18 (Bisimulation de dynamique).

Pour tout diagramme correct $D = (\texttt{links} \cup P, \texttt{free})$, on a $D \rightsquigarrow^* (\texttt{Sol}(P)\texttt{links}, \texttt{Sol}(P)\texttt{free})$.

Preuve.

Par induction sur le nombre d'équations dans P et généralisation sur $\texttt{links}$:

Cas de base On a $P = \varnothing = \texttt{Sol}(P)$.

Donc $D \rightsquigarrow^0 (\texttt{links}, \texttt{free})$ qui est égal à $(\texttt{Sol}(P)\texttt{links}, \texttt{Sol}(P)\texttt{free})$.

Récurrence Par hypothèse d'induction, pour tout $\texttt{links}'$ et $D_n = (\texttt{links}' \cup P, \texttt{free})$ on a

$$D_n \rightsquigarrow^* (\texttt{Sol}(P)\texttt{links}', \texttt{Sol}(P)\texttt{free})$$

avec n équations dans P. Montrons que pour $D_{n+1} = (\texttt{links} \cup (P \cup \{t \doteq u\}), \texttt{free})$ on a

$$D_{n+1} \rightsquigarrow^* (\texttt{Sol}(P \cup \{t \doteq u\})\texttt{links}, \texttt{Sol}(P \cup \{t \doteq u\})\texttt{free})$$

Par associativité de $\cup$, on a $D_{n+1} = ((\texttt{links} \cup \{t \doteq u\}) \cup P, \texttt{free})$. En utilisant l'hypothèse d'induction avec $\texttt{links}' = \texttt{links} \cup P$ on a :

$$D_{n+1} \overset{IH}{\rightsquigarrow} (\texttt{Sol}(P)(\texttt{links} \cup \{t \doteq u\}), \texttt{Sol}(P)\texttt{free})$$

En appliquant une étape de fusion sur $t \doteq u$ on a :

$$\rightsquigarrow (\texttt{Sol}(t \doteq u)\texttt{Sol}(P)\texttt{links}, \texttt{Sol}(t \doteq u)\texttt{Sol}(P)\texttt{free})$$

Par le lemme de décomposition de solution, cela revient à la décomposition d'une solution complète pour $P \cup \{t \doteq u\}$. Donc, avec la commutativité de $\cup$, on obtient :

$$= (\texttt{Sol}(P \cup \{t \doteq u\})\texttt{links}, \texttt{Sol}(P \cup \{t \doteq u\})\texttt{free})$$

à renommage près.

□

Théorème 19 (Équivalence entre actualisation et réduction par fusion).

Pour tout diagramme correct D, on a $D \rightsquigarrow^* (\varnothing, \Downarrow D)$.

Preuve.

C'est une conséquence directe du lemme de bisimulation de dynamique avec $P = \varnothing$. Comme on a aucune liaison restante, $(\varnothing, \Downarrow D)$ repésente un état final pour la réduction par fusion.

□

Corollaire 20 (Etoile induite).

La procédure de réduction par fusion sur un diagramme D produit une étoile c'est-à-dire que si $D \rightsquigarrow^* (\varnothing, D')$ alors D' représente une étoile.

Preuve.

Par équivalence entre actualisation et fusion, dans un tel cas on a $D' = \Downarrow D$ qui est une étoile (car c'est l'application d'une substitution à l'étoile des rayons libres). Donc D' représente bien une étoile.

□

Il existe deux niveaux de confluence :

— Une au niveau de *l'actualisation*

— Une au niveau de la *normalisation*

On rappelle que l'on considère nos opérations à renommage près, c'est-à-dire qu'on renomme nos entités de telle sorte à ce que = soit confondu avec $\approx_\alpha$ lorsque c'est possible.

Théorème 21 (Confluence de l'actualisation).

L'actualisation est confluente.

Preuve.

L'algorithme d'unification qu'on a choisi est tel que l'ordre d'exécution n'a pas d'influence sur le résultat. Donc, tous les choix mènent vers les mêmes substitutions à renommage près pour un problème P posé par un diagramme.

□

Corollaire 22 (Confluence de la fusion).

La réduction par fusion est confluente.

Preuve.

Comme la réduction par fusion est équivalente à l'actualisation qui est confluente, on peut en déduire qu'elle est aussi confluente.

□

On définit une normalisation pour une unique paire de couleur afin de pouvoir exprimer la propriété de confluence dans notre formalisme.

Definition 57 (Actualisation ciblée).

L'**actualisation ciblée** d'un diagramme D notée $\Downarrow_\alpha D$ applique une fusion pour toutes les liaisons $\alpha/\overline{\alpha}$. L'actualisation ciblée retourne un autre diagramme.

Théorème 23 (Etoile induite ciblée).

Soit D un diagramme avec des liaisons de couleur $\alpha_1, ..., \alpha_n$. L'actualisation ciblée $(\Downarrow_{\alpha_1} ... \Downarrow_{\alpha_n})D$ est une étoile.

Preuve.

L'actualisation est un ensemble de réduction par fusion sur une couleur spécifique. La réduction par fusion sur toutes les couleurs est équivalente à une actualisation simple. Donc par le théorème d'étoile induite, on produit bien une étoile.

□

Théorème 24 (Confluence de l'actualisation ciblée).

L'actualisation ciblée est confluente.

Preuve.

Comme la fusion est confluente en générale, on peut choisir de ne fusionner qu'une couleur puis une autre en particulier. Tous les choix sont équivalents. L'actualisation ciblée étant fondée sur la fusion, elle est confluente.

□

Théorème 25 (Confluence de la normalisation).

Les deux procédures de normalisation suivantes sont équivalentes pour deux paires de couleurs α et β :

- On normalise les couleurs une par une (α puis β ou l'inverse)
- On normalise toutes les couleur

Preuve.

La normalisation évalue les constellations par construction de diagrammes. Prenons un diagramme D avec les couleurs α et β. Sans perdre de généralité, on peut choisir d'effectuer une actualisation ciblée sur α puis une autre sur β, réalisant ainsi une actualisation complète. Une normalisation complète peut donc être décomposée en une normalisation ciblée par construction de diagrammes et actualisation d'une seule couleur puis l'autre. On se repose sur la confluence de l'actualisation ciblée.

□

On peut généraliser le raisonnement à plusieurs couleurs.

Corollaire 26 (Unicité de la forme normale).

Une constellation fortement normalisable Σ possède une unique forme normale à renommage près.

Preuve.

La procédure de normalisation pour Σ forme N diagrammes $D_1, ..., D_N$. Par confluence de l'actualisation, chaque diagramme D_i produit une étoile résiduelle unique à renommage près. La forme normale est l'ensemble (donc une structure non-ordonnée) des étoiles résiduelles et produit une constellation (avec des ensembles de variables indépendants) donc la forme normale est unique à renommage près.

□

Annexe 3 - Une étude stellaire de la normalisation (incomplet)

On montre quelques résultats concernant la forme des graphes d'unifiabilité et leur impact sur la normalisation. Un premier résultat est qu'une constellation avec un graphe d'unifiabilité acyclique est nécessairement fortement normalisable. On ne prend pas en compte la saturation. On parle de *multigraphe* mais on utilisera le terme graphe.

Definition 58 (Acyclicité des constellations).

Une constellation Σ est **acyclique** si $\mathcal{U}(\Sigma)$ est acyclique.

Definition 59 (Chemin régulier).

Un **chemin régulier** dans un graphe est un chemin π sans boucle, c'est-à-dire qu'il n'y a pas d'arête $e = (s, s)$ pour un certain sommet s.

Lemme 27 (Génération).

Si π est un chemin régulier dans $\mathcal{U}(\Sigma)$ pour une constellation Σ alors on peut définir un diagramme $D(\pi)$ de $\mathcal{U}(\Sigma)$.

Preuve.

Par induction sur le nombre d'arêtes de π :

Cas de base On a une arête e. On a bien formé un arbre et donc un diagramme (car π est régulier).

Récurrence On a $\pi = \rho \cdot e_{n+1}$. Par hypothèse d'induction on peut construire $D(\rho)$ à partir duquel on ajoute l'arête e_{n+1} qui a sa source dans $D(\rho)$ et on choisit pour cible une nouvelle occurrence d'étoile étiquetée par la cible de e_{n+1}. On obtient bien un arbre (car $D(\rho)$ est un arbre et qu'on a ni déconnecté le graphe ni formé de cycle).

□

Lemme 28 (Simulation de chemins).

Soit D un diagramme de $\mathcal{U}(\Sigma)$ qui est acyclique. Si $\pi_n = e_1 \cdot ... \cdot e_n$ est un chemin n dans D alors $f(\pi_n) = f(e_1) \cdot ... \cdot f(e_n)$ est un chemin de taille n dans $\mathcal{U}(\Sigma)$.

Preuve.

Par induction sur la taille d'un chemin π dans G :

Cas de base On a $\pi = e$ où e est une arête. Par définition des diagrammes, cette arête est aussi dans $\mathcal{U}(\Sigma)$.

Récurrence Supposons, par hypothèse d'induction, qu'un chemin $\pi_n = e_1 \cdot ... \cdot e_n$ de taille n dans D induise un chemin $f(\pi_n) = f(e_1) \cdot ... \cdot f(e_n)$ de taille n dans $\mathcal{U}(\Sigma)$. Montrons que

c'est le cas pour un chemin $\pi_{n+1} = \pi_n \cdot e_{n+1} = e_1 \cdot ... \cdot e_n \cdot e_{n+1}$ de taille $n+1$ dans D. On a donc un certain nombre d'arêtes entre e_n et e_{n+1}.

— Soit e_{n+1} a le même label qu'un certain e_k avec $1 \leq k \leq n$ et dans ce cas cela induit un cycle $f(e_k), ..., f(e_{n+1})$ dans $\mathcal{U}(\Sigma)$ ce qui est contradictoire (car $\mathcal{U}(\Sigma)$ est acyclique) donc ce cas ne peut pas se produire.

— Sinon, tous les labels de π_{n+1} sont disjoints et donc cela forme bien un chemin élémentaire $f(\pi_{n+1}) = f(\pi_n) \cdot f(e_{n+1})$ de taille $n+1$ en utilisant le chemin de taille n de l'hypothèse d'induction.

□

Théorème 29 (Finitude).

Si $\mathcal{U}(\Sigma)$ est acyclique alors il induit un nombre fini de diagrammes.

Preuve.

Par le lemme de simulation de chemin, la taille des chemins dans un diagramme de $\mathcal{U}(\Sigma)$ est bornée par celle des chemins de $\mathcal{U}(\Sigma)$. Comme $\mathcal{U}(\Sigma)$ est acyclique, il a un nombre fini de chemin. Donc c'est aussi le cas de tout diagramme D induit. On ne peut donc formé qu'un nombre fini de diagrammes car chaque arbre de $\mathcal{U}(\Sigma)$ est caractérisé par un ensemble d'arêtes de $\mathcal{U}(\Sigma)$.

□

Corollaire 30 (Normalisation forte).

Si une constellation Σ est acyclique alors elle est fortement normalisable.

Preuve.

Par le théorème de finitude, comme Σ est acyclique, son graphe d'unifiabilité $\mathcal{U}(\Sigma)$ induit un nombre fini de diagrammes et donc en particulier un nombre fini de diagrammes corrects.

□

Une remarque intéressante est que la présence de cycle est liée au potentiel infini du nombre de diagramme mais ne l'induit pas nécessairement.

Corollaire 31 (Infinitude).

Si une constellation Σ est cyclique alors elle engendre une infinité de diagrammes D.

Preuve.

Une constellation cyclique possède un graphe d'unifiabilité avec un nombre de chemin arbitraire donc on peut former une infinité de diagrammes grâce au lemme de génération.

□

Théorème 32 (Infinitude annulée).

Il existe au moins une constellation Σ cyclique et fortement normalisable.

Preuve.

On peut choisir la constellation suivante : $[\![\alpha.x, \beta.x]\!] + [\![\overline{\alpha}.f(y), \overline{\beta}.g(y)]\!]$. La constellation possède bien un cycle de taille 2 mais seuls les diagrammes de taille 1 sont corrects. Tous les diagrammes de taille 2 (il n'y en a qu'un) sont incorrects. Donc la constellation est fortement normalisable.

□

Annexe 4 - Modèles de MLL par biorthogonalité

On peut définir un modèle de MLL par biorthogonalité en nous inspirant de l'orthogonalité des entités en ludique [Gir01] et des constructions catégoriques par *tight double glueing* [HS03] avec les éléments suivants :

Dessein Une entité actrice de l'interaction associé à des adresses (aussi appelés *lieux*).

Interaction Une notion d'interaction notée $::$ (qui agit sur un certain lieu) qui va connecter deux desseins pour en produire un autre.

Orthogonalité Une notion d'orthogonalité symétrique qui oppose les desseins et les sépare en deux classes selon un point de vue subjectif/arbitraire.

Cohérence Des propriétés qui assurent la cohérence de l'interaction comme *l'associativité* de l'interaction, et une *propriété d'adjonction* qui va nous permettre de dissocier le comportement du connecteur $\parr$ et $\otimes$.

Definition 60 (Orthogonalité).

On note $\perp$ la relation binaire d'**orthogonalité** entre deux desseins et

$$\mathbf{A}^{\perp} = \{b \mid \forall a \in \mathbf{A}. a \perp b\}$$

Definition 61 (Comportement).

Un **comportement** est un ensemble $\mathbf{A}$ de desseins tel que $\mathbf{A} = \mathbf{A}^{\perp\perp}$

Definition 62 (Tenseur).

Soient $\mathbf{A}, \mathbf{B}$ deux comportements. On définit le tenseur par :

$$\mathbf{A} \otimes \mathbf{B} = \{a \oslash b \mid a \in \mathbf{A}, b \in \mathbf{B}\}^{\perp\perp}$$

où $a \oslash b$ est une opération fixée d'avance.

Definition 63 (Flèche linéaire).

Soient $\mathbf{A}, \mathbf{B}$ deux comportements. On définit la flèche linéaire par :

$$\mathbf{A} \multimap \mathbf{B} = \{f \mid \forall a \in \mathbf{A}, f :: a \in \mathbf{B} \text{ et } f \perp a\}^{\perp\perp}$$

Axiome 33 (Adjonction).

Soient a, b, c des desseins. On a $a \perp (b \otimes c)$ si et seulement si $a \perp b$ et $a :: b \perp c$.

Théorème 34 (Traduction de flèche linéaire).

Soient $\mathbf{A}, \mathbf{B}$ deux comportements. L'adjonction implique :

$$\mathbf{A} \multimap \mathbf{B} = (\mathbf{A} \otimes \mathbf{B}^{\perp})^{\perp}$$

Preuve.

Par définition, $f \in \mathbf{A} \multimap \mathbf{B}$ si et seulement si pour tout $A \in \mathbf{A}$, $f :: a \in \mathbf{B}$ avec $f \perp a$. Comme tout comportement est orthogonal à un autre on a pour tout $a \in \mathbf{A}$ et pour tout $b' \in \mathbf{B}^\perp$, $f :: a \perp b'$. Par adjonction, $f \perp a \otimes b'$. Par conséquent, $f \in (\mathbf{A} \otimes \mathbf{B}^\perp)^\perp$.

□

En nous aidant de ces modèles par biorthogonalité on peut ensuite construire des catégories $*$-autonomes nous fournissant un modèle de MLL. On peut retrouver une telle reconstruction dans les graphes d'interaction de Seiller [Sei12a].

Annexe 5 - Modèle d'étoiles

Definition 64 (Execution).

Soient Σ_1 et Σ_2 deux constellations. Leur exécution est définie par $\Sigma_1 :: \Sigma_2 = \mathtt{Ex}^\star(\Sigma_1 \cup \Sigma_2)$.

Definition 65 (Type).

Un **type** est un ensemble de constellations $\mathbf{A}$ tel que il existe un ensemble de constellations B tel que $\mathbf{A} = B^\perp$.

Lemme 35 (Inclusion).

Pour tout ensemble de constellations A, on a $A \subseteq A^{\perp\perp}$.

Preuve.

Soit $\Sigma \in A$. Par définition, pour tout $\Sigma' \in A^\perp$, $\Sigma \perp \Sigma'$. Par conséquent $\Sigma \in A^{\perp\perp}$ par définition de $A^{\perp\perp}$.

□

Lemme 36 (Triple orthogonalité).

Pour tout ensemble de constellations A, on a $A^{\perp\perp\perp} = A^\perp$.

Preuve.

($A^\perp \subseteq A^{\perp\perp\perp}$) Conséquence du lemme d'inclusion.

($A^{\perp\perp\perp} \subseteq A^\perp$) Soit $\Sigma \in A^{\perp\perp\perp}$. Par définition, il est orthogonal à tout élément de $A^{\perp\perp}$. Or, on a $A \subseteq A^{\perp\perp}$ par le lemme d'inclusion. Donc Σ est orthogonal à tout élément de A et par conséquent $\Sigma \in A^\perp$ par définition de $A^\perp$.

□

Théorème 37 (Clôture par biorthogonal).

Soit $\mathbf{A}$ un ensemble de constellations. On a $\mathbf{A} = \mathbf{A}^{\perp\perp}$ si et seulement si $\mathbf{A}$ est un type.

Preuve.

($\Rightarrow$) Supposons qu'on a $\mathbf{A} = \mathbf{A}^{\perp\perp}$. Donc on a $B = \mathbf{A}^\perp$ tel que $\mathbf{A} = (\mathbf{A}^\perp)^\perp = B^\perp$.

($\Leftarrow$) Soit $B^\perp$ tel que $\mathbf{A} = B^\perp$. Montrons qu'on a $\mathbf{A} = \mathbf{A}^{\perp\perp}$. Par le lemme de triple orthogonalité sur B, on a $B^{\perp\perp\perp} = B^\perp$, donc $\mathbf{A} = \mathbf{A}^{\perp\perp}$.

□

On définit un modèle de MLL par orthognalité avec les constellations comme desseins, l'exécution comme interaction et la terminaison de l'exécution comme orthogonalité.

Definition 66 (Tenseur de constellations).

Soient Σ_1 et Σ_2 deux constellations. On définit le tenseur de constellations par :

$$\Sigma_1 \otimes \Sigma_2 := \texttt{left}(\Sigma_1) \cup \texttt{right}(\Sigma_2)$$

où `left` et `right` s'appliquent aux rayons colorés avec :

$$\texttt{left}(\alpha.t) = \alpha_l.t \qquad \texttt{right}(\alpha.t) = \alpha_r.t \qquad \text{sinon} \qquad \texttt{left}(t) = \texttt{right}(t) = t$$

C'est-à-dire que Σ_1 et Σ_2 sont recolorées de sortent à ce que leurs couleurs soient disjointes. Cela nous permet d'avoir une exclusion des interactions possibles.

Definition 67 (Orthogonalité).

Soient Σ_1 et Σ_2 deux constellations. On a $\Sigma_1 \perp \Sigma_2$ si $\Sigma_1 :: \Sigma_2$ est fortement normalisable.

Théorème 38 (Associativité de l'exécution).

Soient $\Sigma_1, \Sigma_2, \Sigma_3$ des constellations. On a $(\Sigma_1 :: \Sigma_2) :: \Sigma_3 = \Sigma_1 :: (\Sigma_2 :: \Sigma_3)$.

Preuve.

La preuve découle naturellement du théorème d'équivalence des focalisations (qui est une forme de confluence, comme l'associativité).

Si on a $\texttt{colors}(\Sigma_1) \cap \texttt{colors}(\Sigma_2) \cap \texttt{colors}(\Sigma_3) = \varnothing$ **:** il n'y a pas de liaisons. On a $(\Sigma_1 :: \Sigma_2) :: \Sigma_3 = \texttt{normal}(\Sigma_1 \cup \Sigma_2 \cup \Sigma_3) = \Sigma_1 :: (\Sigma_2 :: \Sigma_3)$.

Si on a $\texttt{colors}(\Sigma_1) \cap \texttt{colors}(\Sigma_2) \cap \texttt{colors}(\Sigma_3) \neq \varnothing$ **:** par le théorème d'équivalences des focalisations, les deux exécutions $(\Sigma_1 :: \Sigma_2) :: \Sigma_3$ et $\Sigma_1 :: (\Sigma_2 :: \Sigma_3)$ peuvent être décomposés en séries d'actualisations ciblées. Comme l'ordre des actualisations ciblées peut être choisi arbitrairement sans impact sur le résultat alors on peut choisir de commencer soit par des fusions sur $\texttt{colors}(\Sigma_1) \cap \texttt{colors}(\Sigma_2)$ ou sur $\texttt{colors}(\Sigma_2) \cap \texttt{colors}(\Sigma_3)$. Donc $(\Sigma_1 :: \Sigma_2) :: \Sigma_3$ et $\Sigma_1 :: (\Sigma_2 :: \Sigma_3)$ ont la même forme normale.

Sinon : soit on a seulement $\texttt{colors}(\Sigma_1) \cap \texttt{colors}(\Sigma_2) = \varnothing$ ou $\texttt{colors}(\Sigma_2) \cap \texttt{colors}(\Sigma_3) = \varnothing$. Prenons le premier cas sans perdre de généralité. La constellation Σ_1 est indépendante de Σ_2 donc on peut commencer par $\Sigma_2 :: \Sigma_3$ pour atteindre le même résultat.

□

Théorème 39 (Adjonction).

Soient $\Sigma_1, \Sigma_2, \Sigma_3$ des constellations. On a $\Sigma_1 \perp (\Sigma_2 \otimes \Sigma_3)$ si et seulement si $\Sigma_1 \perp \Sigma_2$ et $(\Sigma_1 :: \Sigma_2) \perp \Sigma_3$.

Preuve.

Preuve à compléter. La preuve devrait découler de la confluence de la normalisation de constellation.

□

Il reste à montrer qu'on peut former une catégorie $*$-autonome $\mathbf{Star}_{MLL}$. On peut, pour cela, s'inspirer d'construction plus détaillée où on construit une catégorie $*$-autonome à travers un modèle de MLL par orthogonalité pour les graphes d'interaction [Sei12a].

Annexe 6 - Des étoiles aux flots (expérimental)

On cherche à simuler les flots à partir des étoiles. Les flots ont une polarité, c'est-à-dire qu'on ne connecte que les têtes et les queues. On peut représenter un flot par une étoile à deux couleurs $(t \leftarrow u)^\star = [\![-t, +u]\!]$ où $+$ et $-$ sont deux couleurs duales. On peut étendre cette traduction aux câblages.

Cependant, la dynamique des flots est différente de celle des étoiles. Les flots sont plus libres que les étoiles car on ne leur impose pas de symétrie. En particulier le produit de flot est orienté. On peut, par contre, montrer qu'on peut simuler la notion d'exécution qu'on utilise en logique.

Théorème 40 (Simulation).

Soit F et G deux câblages. On a $\mathtt{Ex}_f(F, G) = \mathtt{Ex}^\star(F^\star \cup G^\star)$.

On peut, à l'inverse, imaginer s'il est possible de simuler les étoiles avec une extension des flots. En particulier, il serait idéal d'avoir un formalisme aussi expressif sans être soumis à une symétrie. On peut par exemple imaginer une idée de lieur de variable entre des flots.

Annexe 7 - Critère de correction

Théorème 41 (Correspondance comportementale).

Soit π une structure de preuve de conclusion Γ et $V + t$ un test $t \in G(\pi)$ de π complété par les axiomes $V(\pi)$:

1. Si $V(\pi) + t$ est cyclique, $\alpha.\|V(\pi)\| \cup \|t\|$ a une infinité de diagrammes saturés.
2. Si $V(\pi) + t$ est non-connexe, alors on peut construire l'unique diagramme saturé sur $\alpha.\|V(\pi)\| \cup \|t\|$ qui s'actualise en $[\![p_{\Gamma'}]\!]$ avec $\Gamma' \subsetneq \Gamma$.
3. Si $V(\pi) + t$ est un arbre alors on peut construire l'unique diagramme sur $\alpha.\|V(\pi)\| \cup \|t\|$ qui s'actualise en $[\![p_\Gamma]\!]$.

Preuve.

Preuve à compléter.

□

Corollaire 42 (Simulation de correction).

Soit π une structure de preuve de MLL. On a π est correct si et seulement si π satisfait le critère de correction stellaire.

Preuve.

Preuve à compléter.

□

Annexe 8 - Le maillage conceptuel de Girard (expérimental)

Le programme repose sur un maillage conceptuel [Gir16a] qui réorganise la relation entre logique et calcul qui sont couramment entremêlés.

	Analytique/Réponses	**Synthétique/Questions**
Explicite	Constat	Usine
Implicite	Performance	Usage

Ce modèle conceptuel tire son inspiration de l'epistémologie de Kant et en propose une mise à jour tenant compte de la compréhension moderne de la logique et de l'introduction du calcul par l'informatique.

Le point aveugle

Une raison d'être de cette organisation provient de l'idée de « point aveugle », l'idée que l'acquisition de la connaissance est obstruée par les observations premières et les préconceptions que l'on a d'un sujet. C'est une idée qui émerge du concept de *rupture epistémologique* (qu'on peut associer à Bachelard puis Althusser). Ainsi, pour accéder à un contenu plus "réel" de la logique, il faut identifier des obstacles et procéder à un passage des préjugés au réel et de l'expérience à la raison. C'est dans cette perpétuelle remise en question qu'évolue un domaine scientifique.

En particulier, la logique a la particularité d'être un domaine fortement biaisé par notre conception du monde, un biais dont il faut se défaire. Ainsi, la logique qui émerge de nos observations du réel (vérité, formules, connecteurs logiques) trouve une morphologie du raisonnement à travers les preuves (arborescentes) puis les réseaux de preuves (topologiques) et enfin les flots/étoiles (asynchrones).

Il semblerait que nous n'ayons accès à rien de plus tangible que la syntaxe (la sémantique étant elle-même une syntaxe qui nous échappe). L'idée de la Syntaxe Transcendantale est donc de trouver un espace syntaxique qui ne se réfère à rien d'autre que lui-même. Cette internalisation nous est permise par une interaction calculatoire et symétrique entre différentes entités vivant dans un même espace. Ce changement de paradigme nous permet entre d'autre d'étudier la logique sous un nouveau point de vue plus riche.

Analytique / Réponses

Transparence et opacité Ce que Girard appelle *analytique* correspond à un espace calculatoire libre où résultat (constat) et programme (performance) cohabitent : il ne doit pas y avoir d'opérations cachées. Par exemple l'évaluation du λ-caclul pur (avec le couple forme normale / rédex) cache des opérations complexes comme le montre son étude par la logique linéaire. Il est donc important de trouver un espace analytique satisfaisant utilisé comme matériau brut pour reconstruire la logique avec le maximum de transparence. Girard appelle *épistate* une généralisation calculatoire des preuves vivant dans un espace purement analytique.

Subjectivité et objectivité L'informatique nous montre que les programmes peuvent être des résultats. Par conséquent, constats et performances sont considérés comme des réponses, sans signification, sans engagement. Cela amène à deux constatations : (1) les réponses portent une part de subjectivité : on décide de voir un programme comme résultat ou comme exécution, (2) l'absence d'un fichier dans un ordinateur ne signifie en rien l'absence

du fichier recherché, il peut être caché ou avoir une extension différente, la "réponse" est donc indépendante de toute conception de signification ce qui la rend objective (la subjectivité est effacée si on se place dans un espace où constat et performance sont distingués).

Unification Il existe une multitude d'espaces analytiques. Par exemple, l'espace analytique standard de l'informatique est donné par le codage binaire codant à la fois les résultats et les programmes qui y mènent [Gir13b]. L'étude de la géométrie de l'interaction s'est stabilisé sur un espace analytique basé sur l'unification de Herbrand qui a donné le modèle des flots [Gir95] puis qui a été mis à jour vers le modèle d'étoiles de la Syntaxe Transcendantale pour permettre des considérations logiques plus ambitieuses (prise en compte des types/formules et introduction des tests).

La nature de l'analyticité Ce que Girard entend par un "bon espace analytique" n'est pas parfaitement clair. Tous les modèles de calcul possèdent une part d'analyticité mais ne se valent pas. De plus, il n'est pas évident que l'unifcation fournit un espace "purement" analytique malgré qu'on en ait une intuition vague par le fait qu'il calcule par interaction entre des entités duales (termes unifiables). Une compréhension plus mature (topologique ?) pourrait probablement fournir un éclaircissement (par exemple, la présentation de l'unification par graphe de terme décrit l'unification comme une manipulation de pointeurs). Les graphes d'interaction [Sei12b] semblent fournir une réponse plus satisfaisante puisque l'évaluation est immédiatement induite par la juxtaposition de deux graphes. Est-ce que l'analyticité est une histoire de manipulation purement explicite et spatiale ?

Synthétique / Questions

Emergence du sens Le sens émerge par les conventions et l'usage d'entités : Girard propose une dichotomie entre sens d'usine et sens d'usage qui sont deux facette du sens. Le synthétique est à la logique ce que l'analytique est au calcul : il type le calcul. Dans les étoiles et constellations cela correspond à connecter le gabarit à un véhicule. La connexion elle-même est analytique (calculatoire) mais leur réunion devient synthétique i.e subjectivement engagée puisqu'une épistate est considérée comme réponse à une formule logique que Girard appelle *dichologie*. La logique classique répond aux questions dans un espace booléen et la logique classique dans un espace de preuves. Il y a des réponses plus satisfaisantes que d'autres mais c'est principalement le problème de la *certitude* qui les sépare.

Usine Le sens d'usine est celui qui pose les labels et l'usage prévu. Girard se réfère à l'industrie automobile. Un véhicule porte une appelation après avoir passé un certain nombre de tests certifiant son usage. Pour les réseaux de preuve cela correspond aux graphes de test que l'on connecte aux axiomes pour vérifier le critère de correction de Danos-Regnier. Ce sont des tests qui sont explicites et qui correspondent au synthétique à posteriori de Kant (la certification par test est empirique). Dans l'idéal, on souhaite un nombre fini de tests d'usine.

Usage Le sens comme usage est celui auquel Wittgenstein se réfère [Wit09]. Le sens d'usage d'une entité est défini par le potentiel d'interaction avec d'autres entités du même espace. Chaque entité est donc vu comme un "test d'usage" permettant de définir le sens d'autres entités de façon mutuelle. Le test d'une entité impose une opposition sujet/objet mais qui est interchangeable (le sujet peut être vu comme objet et vice-versa). Prenons un programme informatique, on peut le définir par son comportement lorsqu'il interagit avec un certain environnement qui à son tour, peut être défini par son comportement selon les programmes qui y sont interprétés. En logique, les connecteurs sont reconstruits selon leur potentiel d'interaction. L'usage d'une entité est souvent imprévisible, dû à l'infinité des interactions

potentielles. En particulier, l'usage peut être détourné et ne pas être conforme à celui qu'on avait prévu.

Typage Le synthétique est fondamentalement une affaire de typage. Un type ou une formule (conformément à la correspondance de Curry-Howard) est un ensemble de tests (qui sont en fait des entités calculatoires comme les autres). Si le type est clos par biorthogonal c'est-à-dire qu'on a $\mathbf{A} = \mathbf{A}^{\perp\perp}$ alors c'est un test d'usage qui peut interagir mutuellement avec les autres. Les tests d'usines, en général, ne sont pas clos par biorthogonal ce qui créer une asymétrie des tests (on teste les graphes de Danos-Regnier sur un véhicule pour définir un réseau de preuve).

Certitude Le problème de la certitude est le suivant : à quel point l'usine certifie l'usage ? [Gir13b] C'est-à-dire, est-ce que par nos définitions (tests d'usine) on capture suffisamment bien l'usage qu'on peut en faire. En termes logiques, comment être sûr qu'une preuve est bien une preuve de ce qu'elle affirme ? Pour les réseaux, la certitude est absolue (*apodictique* pour Girard) pour MLL (car le nombre de tests de Danos-Regnier est fini et l'élimination des coupures plutôt limitée) mais serait seulement raisonnable (*épidictique* pour Girard) pour des fragments plus riches comme la logique du second ordre puisque le nombre de tests est infini mais il semblerait que la certitude soit donnée par un échantillon fini. Ce n'est pas quelque chose que nous avons abordé dans notre travail.

Les liaisons conceptuelles

Constat/Performance La chose qui assure le passage de la performance (l'évaluable) au constat (l'évalué) est la *normalisation forte* c'est-à-dire la terminaison du calcul. Dans le formalisme des étoiles et des constellations, la normalisation diverge sur le nombre de diagrammes produit est infini. Seul la finitude du nombre de diagramme permet d'atteindre un résultat fini.

Usine/Usage La liaison entre usine et usage peut être qualifiée de *déontique*. Il s'agit d'un équilibre entre les définitions (devoirs) et l'utilisation (droits). Les deux ne concordent pas nécessairement. La *normalisation* en déduction naturelle nous assure qu'on peut anihiler les séquences introduction/élimination correspondant à une dualité définition/usage (voir annexe sur la justification des lois logiques). En calcul des séquents, l'usage prend la forme de coupure (usage de lemme) qui permet de connecter deux preuves. L'*élimination des coupures* produit des preuves équivalentes sans coupures démontrant une clôture des droits par les devoirs.

Performance/Usage L'usage possède lui-aussi un contenu calculatoire. En particulier, l'élimination des coupures est une procédure calculatoire. Il s'agit donc de performance. Ce qui assure la cohérence calculatoire de l'élimination des coupure est son *associativité* qui prend la forme de la propriété de Church-Rosser ou de confluence. Dans le formalisme des étoiles et constellations c'est la confluence de la normalisation qui assure une cohérence de l'élimination des coupures.

Les séparations conceptuelles

Constat/Performance Ce qui sépare le constat de la performance est *l'indécidabilité* (qu'on attribue à Church et Turing). C'est-à-dire qu'on ne peut pas réduire l'évaluable à l'évalué. On ne peut pas prédire la terminaison d'un calcul de façon statique/observationnelle.

Usine/Usage Ce qui sépare l'usine et l'usage est *l'indécidabilité* (qu'on attribue à Gödel).

Les deux séparations posent chacunes une forme d'incertitude, l'une dans l'analytique et l'autre dans le synthétique.